

Importing necessary libraries

```
In [1]:

import warnings
import numpy as np
import pandas as pd
import seaborn as sns
import plotly.express as px
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

pd.set_option('display.max_columns',None)
warnings.filterwarnings('ignore')
%matplotlib inline
```

Exploring the dataset

##Read in Collected Data and Process

```
In [2]:

df = pd.read_csv('coords.csv')
```

```
In [3]:

df.head()
```

Out[3]:

	class	x1	y1	z1	v1	x2	y2	z2	v2	x3	y3	z3	v3	x4
0	Standing	0.718350	0.075460	-0.556633	0.999975	0.732501	0.052486	-0.531428	0.999945	0.741256	0.052463	-0.531558	0.999941	0.748299
1	Standing	0.718421	0.078970	-0.424941	0.999977	0.732563	0.055662	-0.404394	0.999949	0.741254	0.055878	-0.404410	0.999945	0.748181
2	Standing	0.718312	0.080733	-0.445973	0.999978	0.732214	0.056878	-0.424561	0.999951	0.740735	0.057149	-0.424588	0.999947	0.747436
3	Standing	0.718029	0.086384	-0.358568	0.999978	0.731237	0.065141	-0.340383	0.999953	0.739484	0.065735	-0.340406	0.999949	0.746261
4	Standing	0.716747	0.091334	-0.408220	0.999980	0.729226	0.070759	-0.389332	0.999955	0.737131	0.071395	-0.389358	0.999952	0.744361

```
In [4]:

df.tail()
```

Out[4]:

	class	x1	y1	z1	v1	x2	y2	z2	v2	x3	y3	z3	v3	x4
2732	Bending	0.640811	0.195959	0.062995	0.997608	0.647675	0.170999	0.025904	0.997666	0.650397	0.168791	0.025838	0.997759	0.653202
2733	Bending	0.660366	0.189389	0.063715	0.997652	0.665472	0.164347	0.031510	0.997711	0.668373	0.161965	0.031377	0.997802	0.671493
2734	Bending	0.664510	0.181341	0.053599	0.997791	0.671355	0.156402	0.023433	0.997850	0.674247	0.154259	0.023285	0.997936	0.677276
2735	Bending	0.667580	0.176877	0.060501	0.997977	0.674071	0.152167	0.029723	0.998031	0.677353	0.149684	0.029578	0.998110	0.680722
2736	Bending	0.667323	0.177113	0.051774	0.998121	0.675047	0.151976	0.024309	0.998174	0.678069	0.149775	0.024133	0.998250	0.681117

```
In [5]:

df.shape
```

Out[5]:

(2737, 133)

```
In [6]:

df.info()
```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2737 entries, 0 to 2736
Columns: 133 entries, class to v33
dtypes: float64(132), object(1)
memory usage: 2.8+ MB

In [7]:

```
df.columns
```

Out[7]:

```
Index(['class', 'x1', 'y1', 'z1', 'v1', 'x2', 'y2', 'z2', 'v2', 'x3',
      ...
      'z31', 'v31', 'x32', 'y32', 'z32', 'v32', 'x33', 'y33', 'z33', 'v33'],
      dtype='object', length=133)
```

In [8]:

```
df.describe()
```

Out[8]:

	x1	y1	z1	v1	x2	y2	z2	v2	x3	y3	
count	2737.000000	2737.000000	2737.000000	2737.000000	2737.000000	2737.000000	2737.000000	2737.000000	2737.000000	2737.000000	273
mean	0.601871	0.237682	-0.412736	0.995316	0.617097	0.202183	-0.402316	0.992715	0.626206	0.202229	-
std	0.123891	0.114531	0.344906	0.015786	0.125478	0.112284	0.332774	0.027338	0.126547	0.111816	
min	0.005324	-0.172922	-1.790220	0.824996	0.058288	-0.195486	-1.853379	0.754173	0.072995	-0.186643	-
25%	0.528982	0.195826	-0.612189	0.997661	0.542261	0.161995	-0.581387	0.997480	0.552655	0.161188	-
50%	0.593356	0.255180	-0.332774	0.999841	0.615322	0.219474	-0.330337	0.999791	0.624976	0.217484	-
75%	0.675936	0.302302	-0.183680	0.999958	0.691495	0.264089	-0.158697	0.999943	0.701853	0.264878	-
max	1.015943	0.802218	0.188332	0.999995	1.022597	0.807167	0.135243	0.999987	1.028872	0.805636	

In [9]:

```
df.describe(include='object')
```

Out[9]:

	class
count	2737
unique	7
top	Sitting
freq	769

In [10]:

```
df['class'].value_counts()
```

Out[10]:

```
Sitting      769
Sitting_left  534
Sitting_right 382
Bending      373
Standing_down 360
Standing_up   172
Standing     147
Name: class, dtype: int64
```

Data Cleaning

Checking duplicates

In [11]:

```
df.duplicated().sum()
```

Out[11]:

0

Checking missing values

In [12]:

```
df.isna().sum()
```

Out[12]:

```
class      0
x1         0
y1         0
z1         0
v1         0
..
v32        0
x33        0
y33        0
z33        0
v33        0
Length: 133, dtype: int64
```

conclusion:no duplicates;no missing values

In [13]:

```
np.unique(df["class"])
```

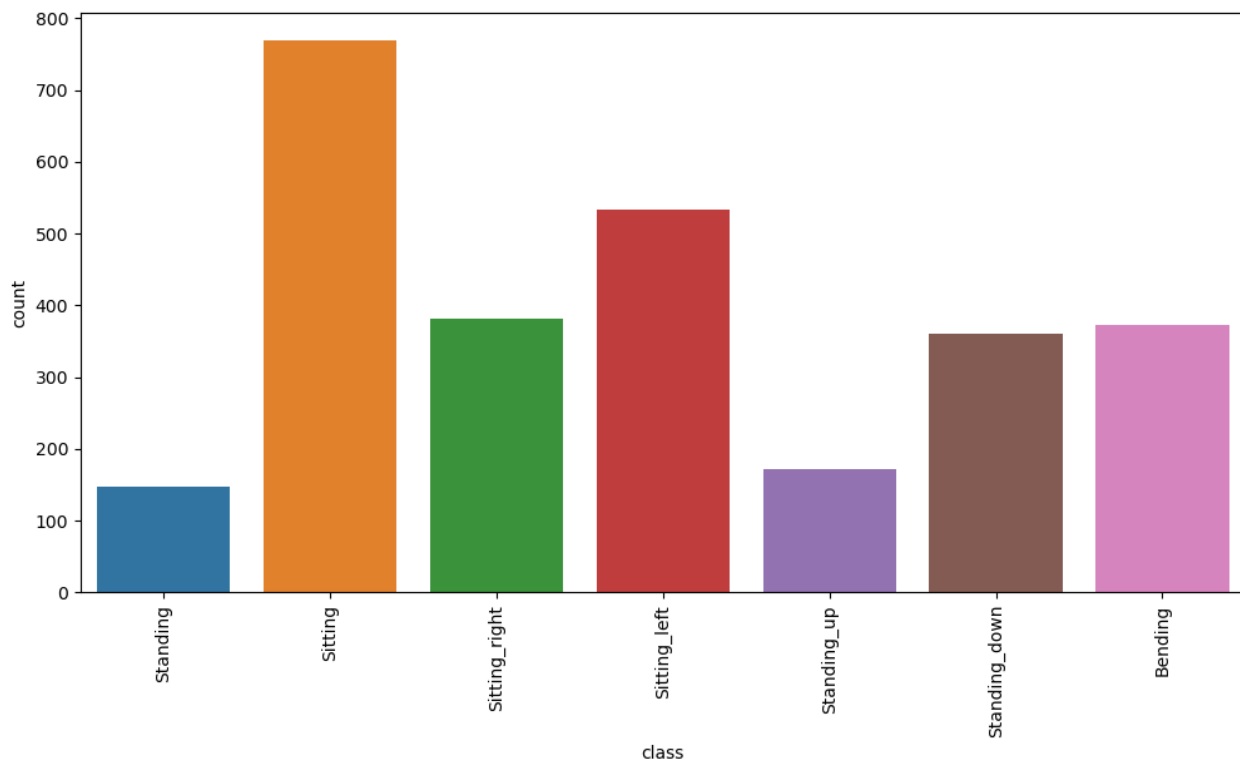
Out[13]:

```
array(['Bending', 'Sitting', 'Sitting_left', 'Sitting_right', 'Standing',
       'Standing_down', 'Standing_up'], dtype=object)
```

Exploratory Data Analysis

In [14]:

```
plt.figure(figsize=(12,6))
axis=sns.countplot(x="class",data=df)
plt.xticks(x=df['class'],rotation='vertical')
plt.show()
```



Checking imbalance

In [15]:

```
fig = px.pie(df, names='class',width=980)
fig.update_layout(
    title={
        'text': "Activities distribution in the data",
        'y':0.95,
        'x':0.40,
        'xanchor': 'center',
        'yanchor': 'top'},
    legend_title ="Activities",
    font=dict(
        family="Arial",
        size=18))
fig.show()
```

In [16]:

```
df.corr()
```

Out[16]:

	x1	y1	z1	v1	x2	y2	z2	v2	x3	y3	z3	v3	x4
x1	1.000000	-0.464598	-0.252970	-0.391983	0.997298	-0.461156	-0.262668	-0.390026	0.994571	-0.458175	-0.262751	-0.392656	0.990958
y1	-0.464598	1.000000	0.015876	0.453549	-0.463903	0.994951	0.025567	0.508963	-0.460404	0.994689	0.025790	0.507354	-0.457403
z1	-0.252970	0.015876	1.000000	0.068363	-0.299513	0.081896	0.995603	0.120126	-0.323670	0.069204	0.995616	0.122987	-0.345212
v1	-0.391983	0.453549	0.068363	1.000000	-0.393111	0.461586	0.070917	0.971662	-0.390542	0.460075	0.071110	0.965109	-0.387249
x2	0.997298	-0.463903	-0.299513	-0.393111	1.000000	-0.464569	-0.309426	-0.395771	0.999405	-0.460769	-0.309507	-0.398562	0.997774
...	...	...	...	...	...	...	...	...	...	...	...	...	...
v32	-0.347009	0.041011	0.602459	0.213480	-0.376075	0.085349	0.596096	0.220790	-0.389742	0.073495	0.596150	0.220696	-0.400084
x33	0.149073	-0.162632	0.181049	-0.201048	0.114664	-0.123227	0.233953	-0.180914	0.095306	-0.115031	0.233728	-0.181476	0.073140
y33	0.176978	0.094154	-0.818957	0.025570	0.228426	0.019657	-0.822275	-0.004302	0.254972	0.030795	-0.822196	-0.000172	0.278465
z33	0.383352	-0.444734	-0.017705	-0.334717	0.371313	-0.443005	-0.045076	-0.322136	0.362383	-0.445111	-0.045197	-0.319355	0.354789
v33	-0.480939	0.154659	0.335633	0.162005	-0.504194	0.193482	0.394914	0.159006	-0.515353	0.196789	0.394819	0.156550	-0.527532

132 rows × 132 columns

In [17]:

```

## Correlation Matrix
sns.set(style="white")

# Compute the correlation matrix
corr = df.corr()
# Generate a mask for the upper triangle
mask = np.zeros_like(corr, dtype=np.bool)
#Return an array of zeros with the same shape and type as a given array.
mask[np.triu_indices_from(mask)] = True
# go through this https://docs.scipy.org/doc/numpy-1.15.1/reference/generated/numpy.triu_indices_from.html

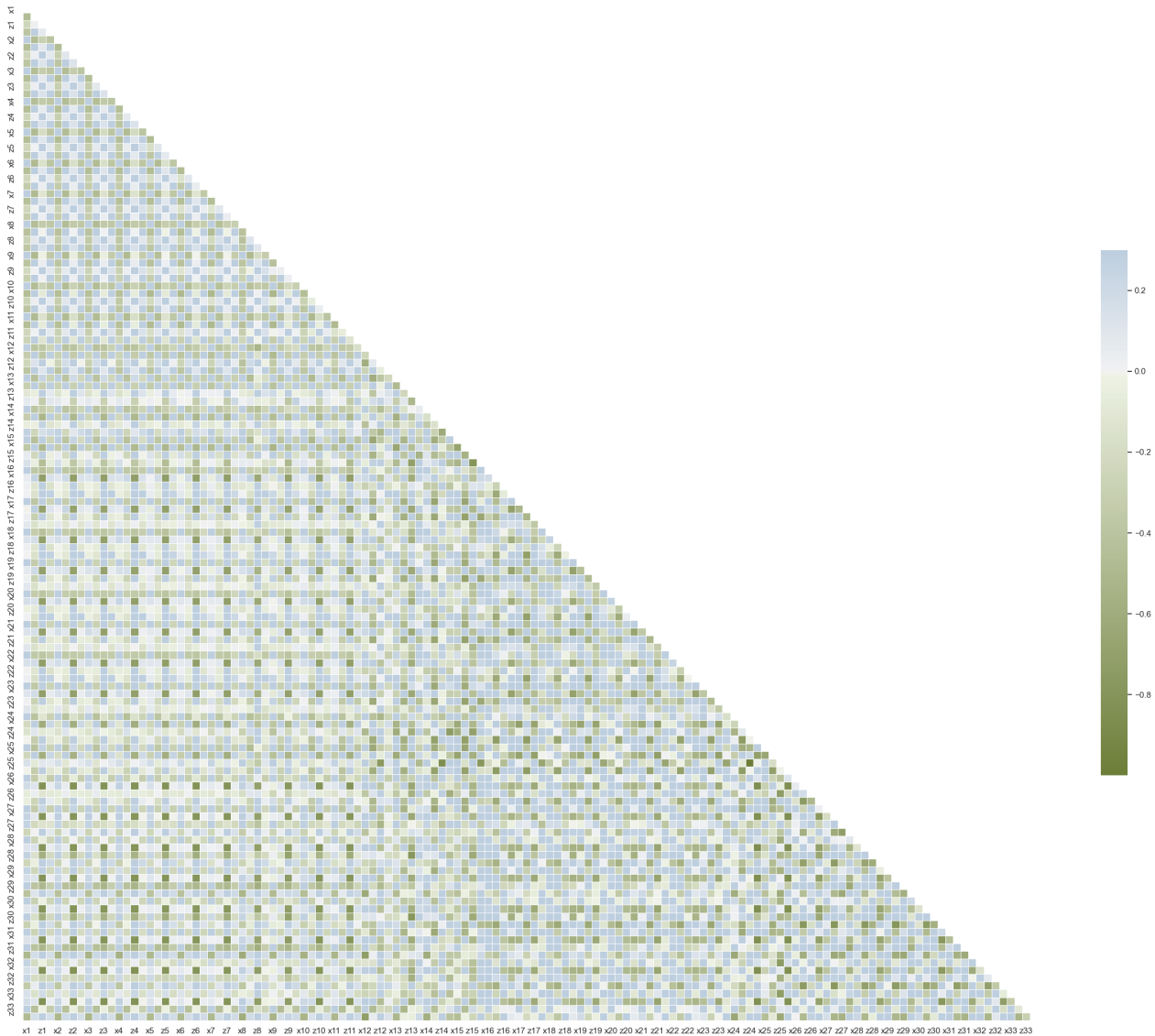
# Set up the matplotlib figure
f, ax = plt.subplots(figsize=(30, 25))

# Generate a custom diverging colormap
cmap = sns.diverging_palette(100, 245, as_cmap=True)#visit for more details: https://seaborn.pydata.org/generated/seaborn.diverging_palette.html
#cmap is short for colormap
# Draw the heatmap with the mask and correct aspect ratio
sns.heatmap(corr, mask=mask, cmap=cmap, vmax=.3, center=0,
            square=True, linewidths=.5, cbar_kws={"shrink": .5})

```

Out[17]:

<AxesSubplot:>



In [18]:



```
plt.figure(figsize=(30, 50))  
plt.title("Heatmap Relation")  
sns.heatmap(df.corr(), annot=True, fmt='.2f')
```

Out[18]:

<AxesSubplot:title={'center':'Heatmap Relation'}>

In [20]:

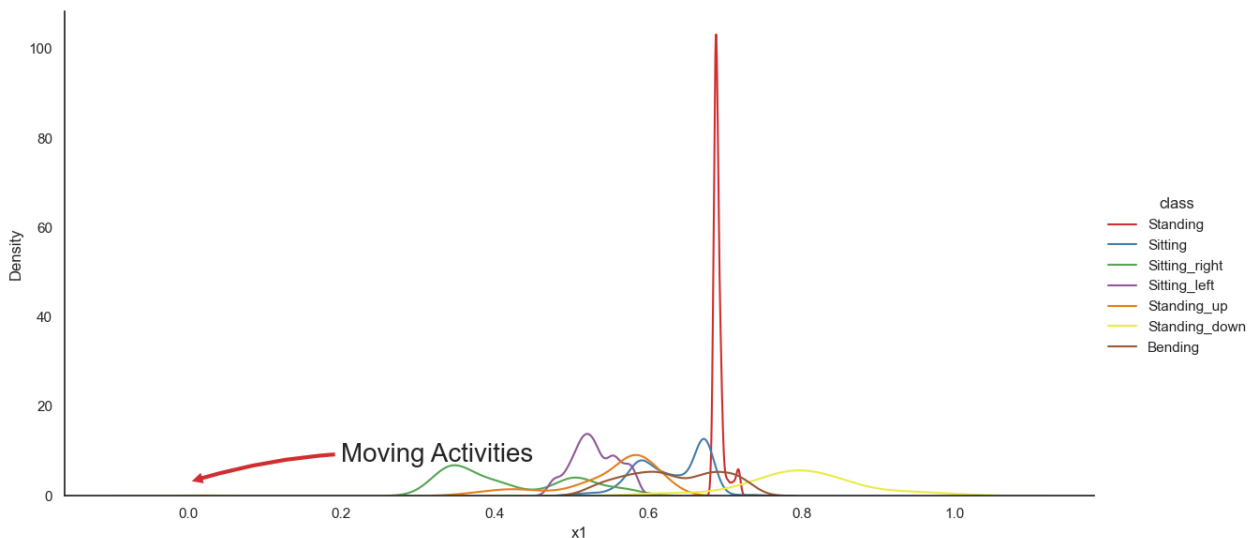
```
# Scaling the data
#from sklearn.preprocessing import StandardScaler
#sc = StandardScaler()
#X_train = sc.fit_transform(X_train)
#X_test = sc.transform(X_test)
```

Visualizing distribution of Stationary and Moving activities

In [21]:

```
sns.set_palette("Set1", desat=0.80)
facetgrid = sns.FacetGrid(df, hue='class', size=6, aspect=2)
facetgrid.map(sns.distplot, 'x1', hist=False)\
    .add_legend()
plt.annotate("Stationary Activities", xy=(-0.92,15), xytext=(-0.5, 15), size=20,\
    va='center', ha='left',\
    arrowprops=dict(arrowsstyle="simple",connectionstyle="arc3,rad=0.1"))

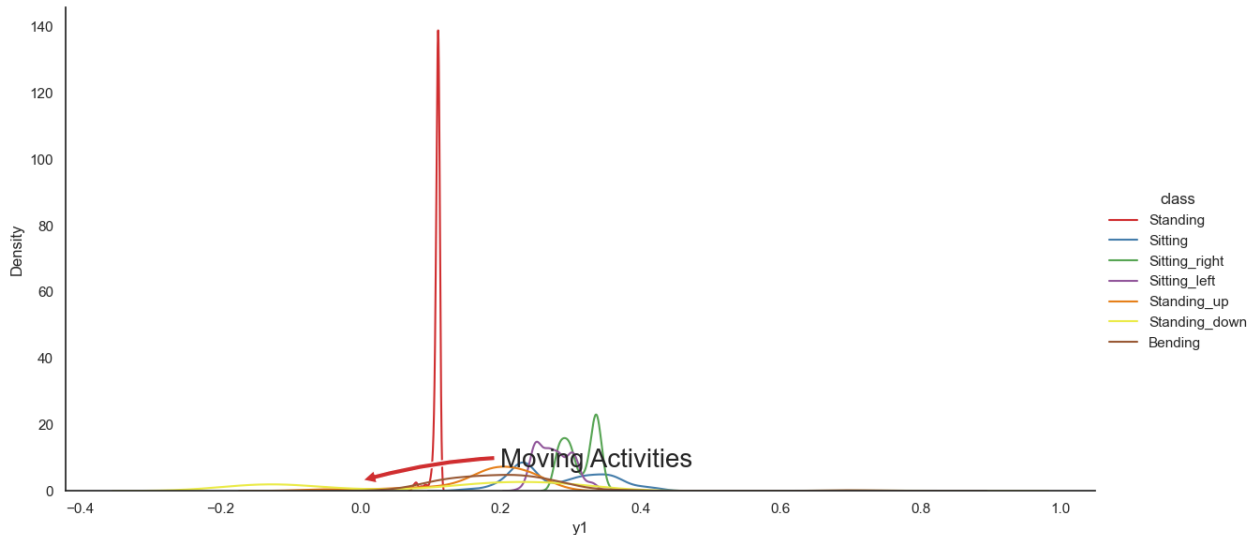
plt.annotate("Moving Activities", xy=(0,3), xytext=(0.2, 9), size=20,\
    va='center', ha='left',\
    arrowprops=dict(arrowsstyle="simple",connectionstyle="arc3,rad=0.1"))
plt.show()
```



In [22]:

```
sns.set_palette("Set1", desat=0.80)
facetgrid = sns.FacetGrid(df, hue='class', size=6, aspect=2)
facetgrid.map(sns.distplot, 'y1', hist=False)\
    .add_legend()
plt.annotate("Stationary Activities", xy=(-0.92,15), xytext=(-0.5, 15), size=20,\
    va='center', ha='left',\
    arrowprops=dict(arrowstyle="simple",connectionstyle="arc3,rad=0.1"))

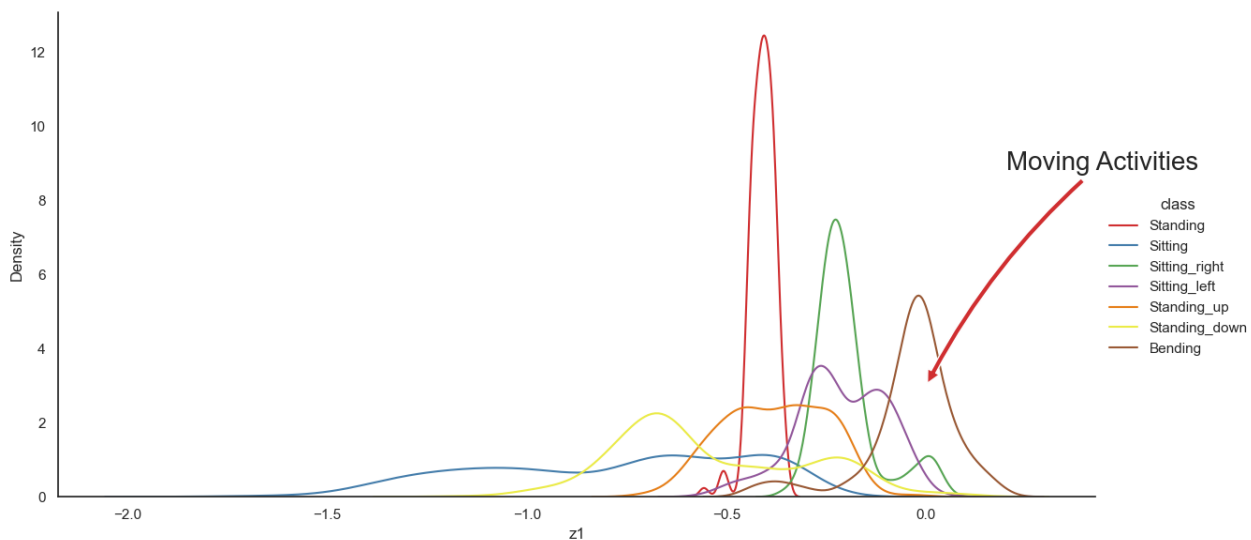
plt.annotate("Moving Activities", xy=(0,3), xytext=(0.2, 9), size=20,\
    va='center', ha='left',\
    arrowprops=dict(arrowstyle="simple",connectionstyle="arc3,rad=0.1"))
plt.show()
```

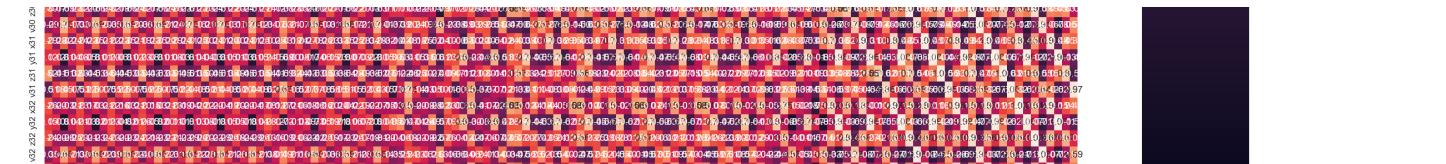
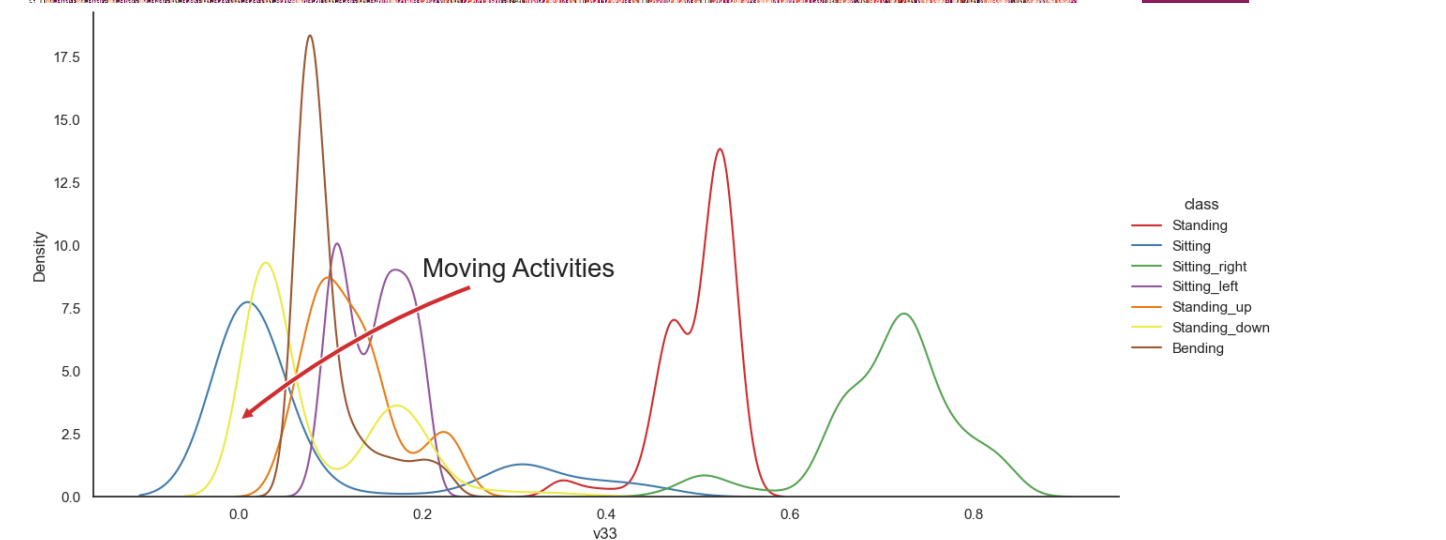
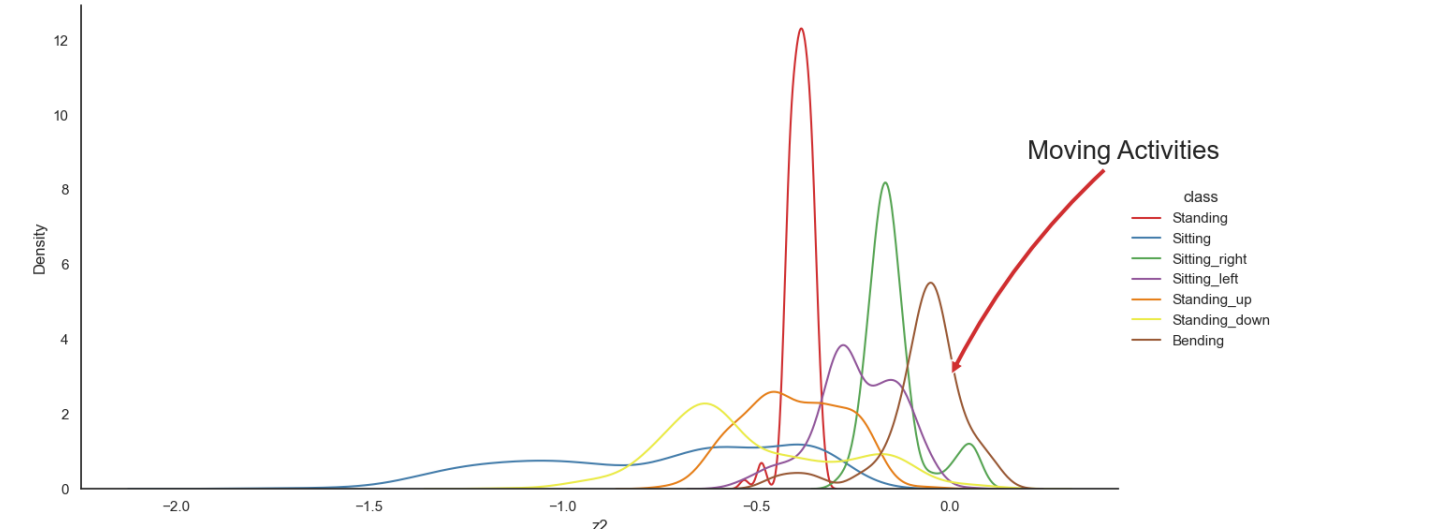
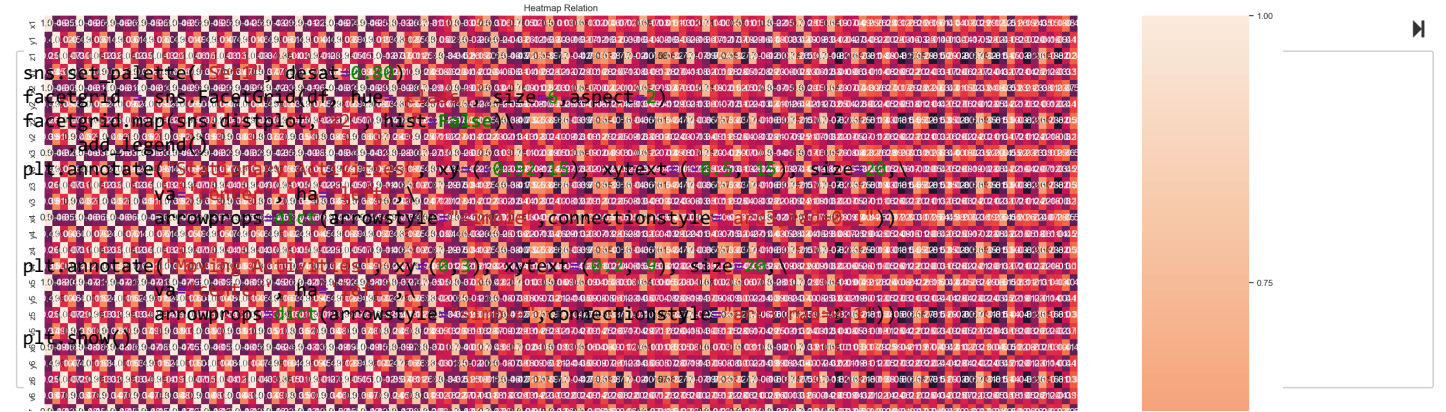


In [23]:

```
sns.set_palette("Set1", desat=0.80)
facetgrid = sns.FacetGrid(df, hue='class', size=6, aspect=2)
facetgrid.map(sns.distplot, 'z1', hist=False)\
    .add_legend()
plt.annotate("Stationary Activities", xy=(-0.92,15), xytext=(-0.5, 15), size=20,\
    va='center', ha='left',\
    arrowprops=dict(arrowstyle="simple",connectionstyle="arc3,rad=0.1"))

plt.annotate("Moving Activities", xy=(0,3), xytext=(0.2, 9), size=20,\
    va='center', ha='left',\
    arrowprops=dict(arrowstyle="simple",connectionstyle="arc3,rad=0.1"))
plt.show()
```



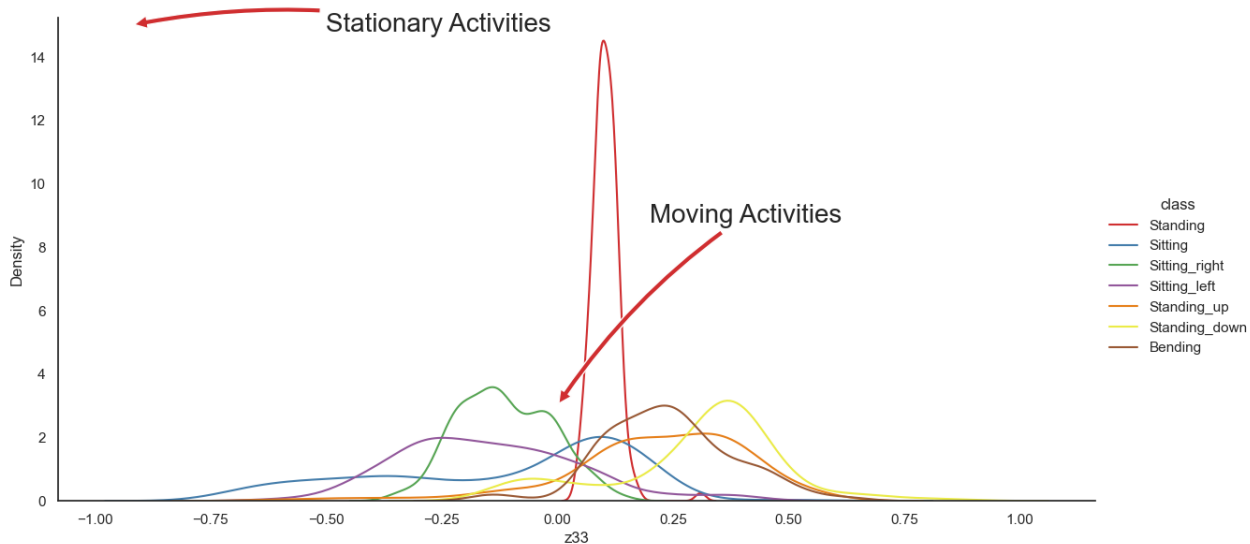


```

sns.set_palette("Set1", desat=0.80)
facetgrid = sns.FacetGrid(df, hue='class', size=6, aspect=2)
facetgrid.map(sns.distplot, 'z33', hist=False)\
    .add_legend()
plt.annotate("Stationary Activities", xy=(-0.92,15), xytext=(-0.5, 15), size=20,\
    va='center', ha='left',\
    arrowprops=dict(arrowstyle="simple",connectionstyle="arc3,rad=0.1"))

plt.annotate("Moving Activities", xy=(0,3), xytext=(0.2, 9), size=20,\
    va='center', ha='left',\
    arrowprops=dict(arrowstyle="simple",connectionstyle="arc3,rad=0.1"))
plt.show()

```



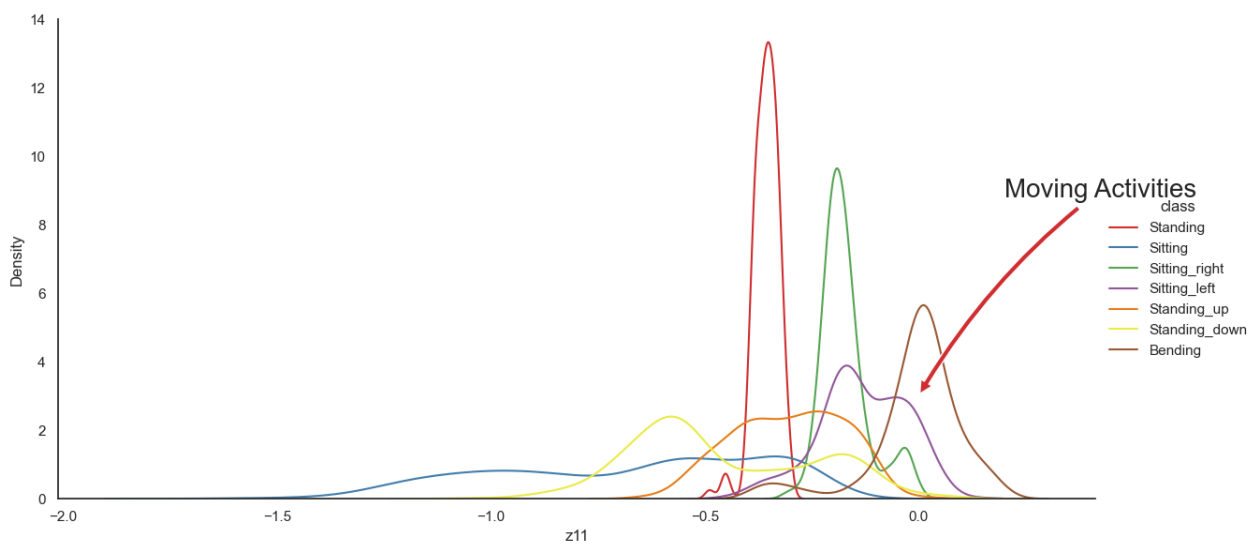
In [27]:

```

sns.set_palette("Set1", desat=0.80)
facetgrid = sns.FacetGrid(df, hue='class', size=6, aspect=2)
facetgrid.map(sns.distplot, 'z11', hist=False)\
    .add_legend()
plt.annotate("Stationary Activities", xy=(-0.92,15), xytext=(-0.5, 15), size=20,\
    va='center', ha='left',\
    arrowprops=dict(arrowstyle="simple",connectionstyle="arc3,rad=0.1"))

plt.annotate("Moving Activities", xy=(0,3), xytext=(0.2, 9), size=20,\
    va='center', ha='left',\
    arrowprops=dict(arrowstyle="simple",connectionstyle="arc3,rad=0.1"))
plt.show()

```



Visualizing data after reducing dimentions with T-SNE

url:# <https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html> (<https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html>)

In [28]:

```
from sklearn.manifold import TSNE
```

In [29]:

```
def perform_tsne(X_data, y_data, perplexities, n_iter=1000, img_name_prefix='t-sne'):  
    for index,perplexity in enumerate(perplexities):  
        # perform t-sne  
        print('\nperforming tsne with perplexity {} and with {} iterations at max'.format(perplexity, n_iter))  
        X_reduced = TSNE(verbose=2, perplexity=perplexity).fit_transform(X_data)  
        print('Done..')  
  
        # prepare the data for seaborn  
        print('Creating plot for this t-sne visualization..')  
        df = pd.DataFrame({'x':X_reduced[:,0], 'y':X_reduced[:,1] , 'label':y_data})  
  
        # draw the plot in appropriate place in the grid  
        sns.lmplot(data=df, x='x', y='y', hue='label', fit_reg=False, size=8,\n                   palette="Set1",markers=['^','v','s','o', '1','2','3'])  
        plt.title("perplexity : {} and max_iter : {}".format(perplexity, n_iter))  
        img_name = img_name_prefix + '_perp_{}_iter_{}.png'.format(perplexity, n_iter)  
        print('saving this plot as image in present working directory...')  
        plt.savefig(img_name)  
        plt.show()  
        print('Done')
```

In [30]:

```
X_pre_tsne = df.drop(['class'], axis=1)
y_pre_tsne = df['class']
perform_tsne(X_data = X_pre_tsne, y_data=y_pre_tsne, perplexities =[5,10,20])
```

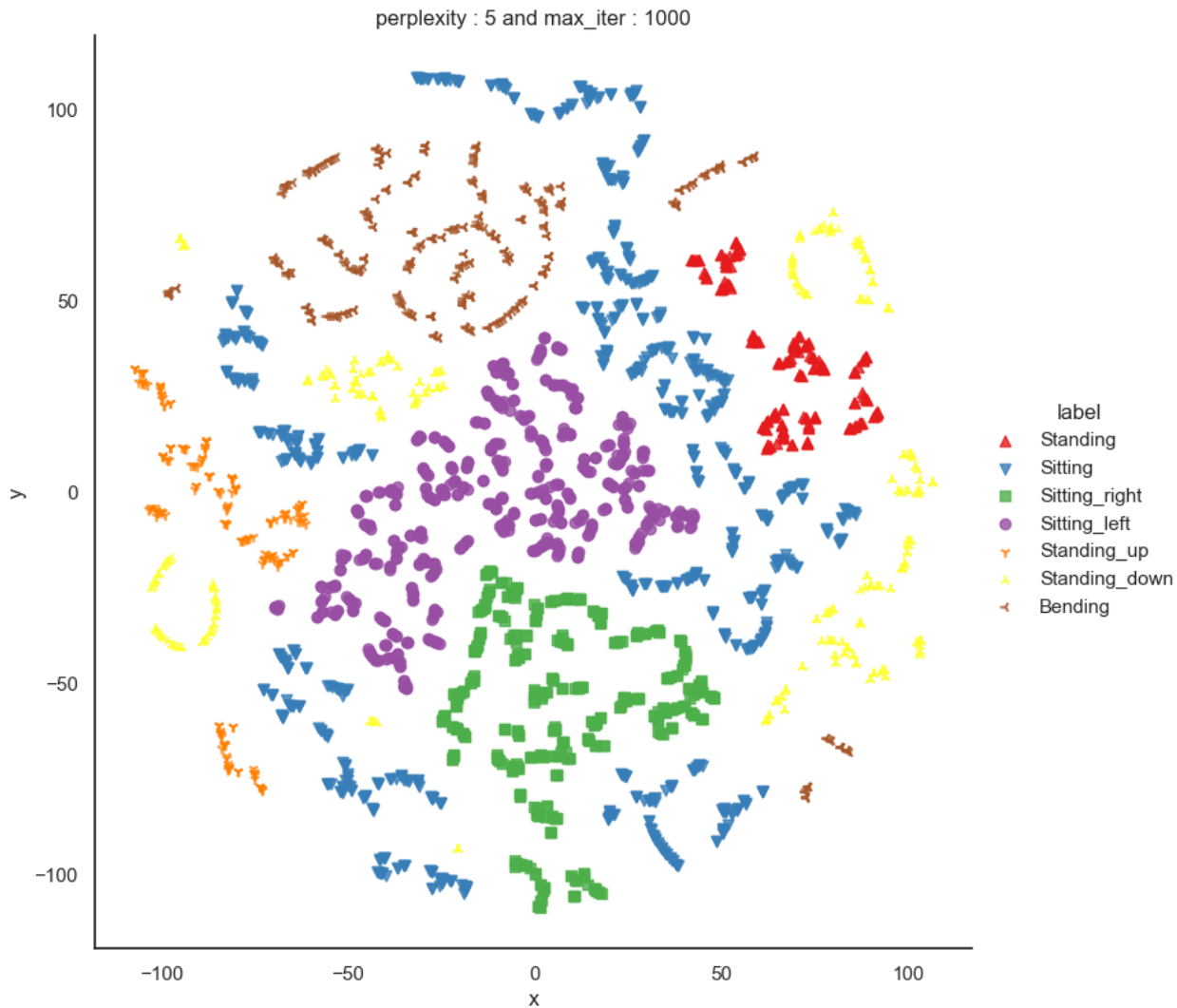
performing tsne with perplexity 5 and with 1000 iterations at max

```
[t-SNE] Computing 16 nearest neighbors...
[t-SNE] Indexed 2737 samples in 0.000s...
[t-SNE] Computed neighbors for 2737 samples in 0.157s...
[t-SNE] Computed conditional probabilities for sample 1000 / 2737
[t-SNE] Computed conditional probabilities for sample 2000 / 2737
[t-SNE] Computed conditional probabilities for sample 2737 / 2737
[t-SNE] Mean sigma: 0.090851
[t-SNE] Computed conditional probabilities in 0.032s
[t-SNE] Iteration 50: error = 96.6785812, gradient norm = 0.1156882 (50 iterations in 0.449s)
[t-SNE] Iteration 100: error = 80.4076767, gradient norm = 0.0573953 (50 iterations in 0.321s)
[t-SNE] Iteration 150: error = 75.0900192, gradient norm = 0.0446729 (50 iterations in 0.285s)
[t-SNE] Iteration 200: error = 72.0084305, gradient norm = 0.0434874 (50 iterations in 0.283s)
[t-SNE] Iteration 250: error = 69.8691788, gradient norm = 0.0325595 (50 iterations in 0.268s)
[t-SNE] KL divergence after 250 iterations with early exaggeration: 69.869179
[t-SNE] Iteration 300: error = 1.7973821, gradient norm = 0.0017620 (50 iterations in 0.313s)
[t-SNE] Iteration 350: error = 1.2489522, gradient norm = 0.0009053 (50 iterations in 0.316s)
[t-SNE] Iteration 400: error = 1.0141302, gradient norm = 0.0006302 (50 iterations in 0.347s)
[t-SNE] Iteration 450: error = 0.8803697, gradient norm = 0.0004665 (50 iterations in 0.350s)
[t-SNE] Iteration 500: error = 0.7918659, gradient norm = 0.0003705 (50 iterations in 0.319s)
[t-SNE] Iteration 550: error = 0.7283411, gradient norm = 0.0003064 (50 iterations in 0.317s)
[t-SNE] Iteration 600: error = 0.6800106, gradient norm = 0.0002739 (50 iterations in 0.338s)
[t-SNE] Iteration 650: error = 0.6421382, gradient norm = 0.0002411 (50 iterations in 0.315s)
[t-SNE] Iteration 700: error = 0.6116804, gradient norm = 0.0002217 (50 iterations in 0.330s)
[t-SNE] Iteration 750: error = 0.5864356, gradient norm = 0.0001910 (50 iterations in 0.349s)
[t-SNE] Iteration 800: error = 0.5653424, gradient norm = 0.0001817 (50 iterations in 0.334s)
[t-SNE] Iteration 850: error = 0.5471560, gradient norm = 0.0001689 (50 iterations in 0.301s)
[t-SNE] Iteration 900: error = 0.5316153, gradient norm = 0.0001602 (50 iterations in 0.331s)
[t-SNE] Iteration 950: error = 0.5190851, gradient norm = 0.0001503 (50 iterations in 0.321s)
[t-SNE] Iteration 1000: error = 0.5075911, gradient norm = 0.0001445 (50 iterations in 0.317s)
[t-SNE] KL divergence after 1000 iterations: 0.507591
```

Done..

Creating plot for this t-sne visualization..

saving this plot as image in present working directory...



Done

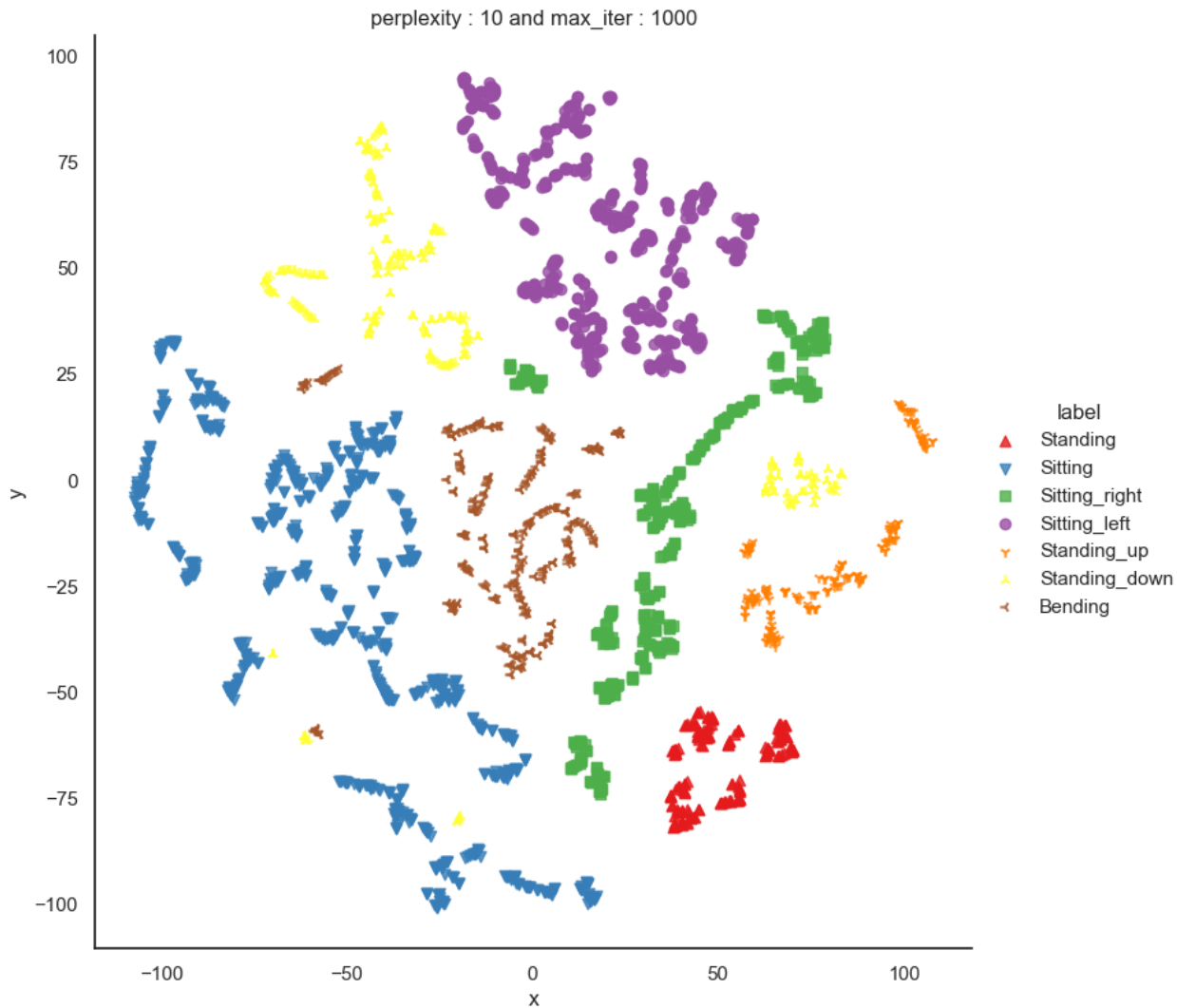
performing tsne with perplexity 10 and with 1000 iterations at max

```
[t-SNE] Computing 31 nearest neighbors...
[t-SNE] Indexed 2737 samples in 0.000s...
[t-SNE] Computed neighbors for 2737 samples in 0.216s...
[t-SNE] Computed conditional probabilities for sample 1000 / 2737
[t-SNE] Computed conditional probabilities for sample 2000 / 2737
[t-SNE] Computed conditional probabilities for sample 2737 / 2737
[t-SNE] Mean sigma: 0.123628
[t-SNE] Computed conditional probabilities in 0.028s
[t-SNE] Iteration 50: error = 84.9811249, gradient norm = 0.0831453 (50 iterations in 0.387s)
[t-SNE] Iteration 100: error = 71.6563721, gradient norm = 0.0400208 (50 iterations in 0.331s)
[t-SNE] Iteration 150: error = 67.4897919, gradient norm = 0.0284429 (50 iterations in 0.293s)
[t-SNE] Iteration 200: error = 65.2220612, gradient norm = 0.0271425 (50 iterations in 0.324s)
[t-SNE] Iteration 250: error = 63.7595024, gradient norm = 0.0175172 (50 iterations in 0.294s)
[t-SNE] KL divergence after 250 iterations with early exaggeration: 63.759502
[t-SNE] Iteration 300: error = 1.4103560, gradient norm = 0.0013316 (50 iterations in 0.299s)
[t-SNE] Iteration 350: error = 0.9040471, gradient norm = 0.0006275 (50 iterations in 0.301s)
[t-SNE] Iteration 400: error = 0.7261609, gradient norm = 0.0004064 (50 iterations in 0.395s)
[t-SNE] Iteration 450: error = 0.6340907, gradient norm = 0.0002967 (50 iterations in 0.353s)
[t-SNE] Iteration 500: error = 0.5776677, gradient norm = 0.0002367 (50 iterations in 0.318s)
[t-SNE] Iteration 550: error = 0.5397732, gradient norm = 0.0002059 (50 iterations in 0.348s)
[t-SNE] Iteration 600: error = 0.5124480, gradient norm = 0.0001806 (50 iterations in 0.350s)
[t-SNE] Iteration 650: error = 0.4927335, gradient norm = 0.0001628 (50 iterations in 0.337s)
[t-SNE] Iteration 700: error = 0.4773909, gradient norm = 0.0001475 (50 iterations in 0.337s)
[t-SNE] Iteration 750: error = 0.4646143, gradient norm = 0.0001386 (50 iterations in 0.368s)
[t-SNE] Iteration 800: error = 0.4544822, gradient norm = 0.0001357 (50 iterations in 0.343s)
[t-SNE] Iteration 850: error = 0.4463650, gradient norm = 0.0001199 (50 iterations in 0.354s)
[t-SNE] Iteration 900: error = 0.4384453, gradient norm = 0.0001173 (50 iterations in 0.337s)
[t-SNE] Iteration 950: error = 0.4318244, gradient norm = 0.0001146 (50 iterations in 0.309s)
[t-SNE] Iteration 1000: error = 0.4257658, gradient norm = 0.0001081 (50 iterations in 0.308s)
[t-SNE] KL divergence after 1000 iterations: 0.425766
```

Done..

Creating plot for this t-sne visualization..

saving this plot as image in present working directory...

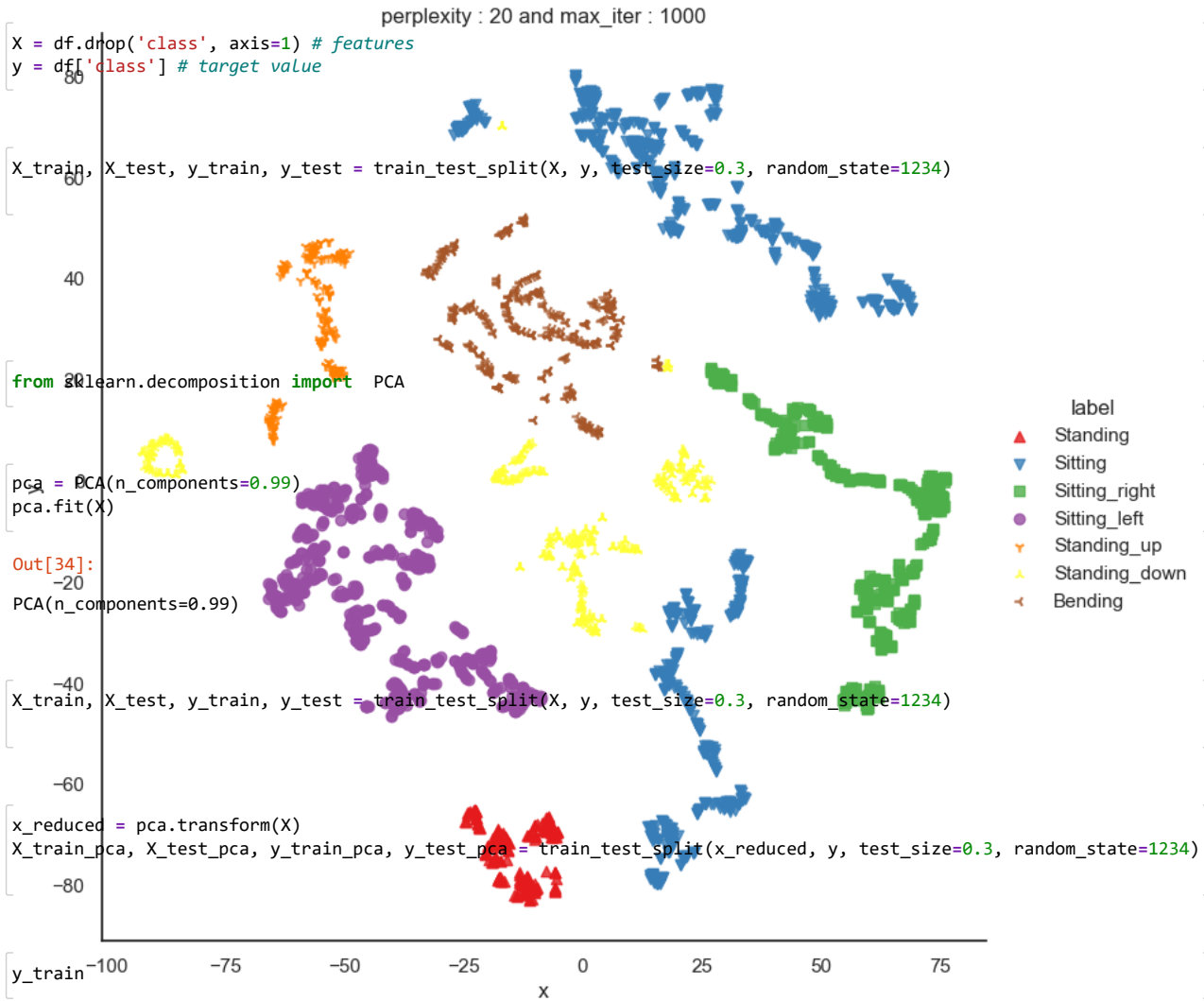


Done

```
performing tsne with perplexity 20 and with 1000 iterations at max
[t-SNE] Computing 61 nearest neighbors...
[t-SNE] Indexed 2737 samples in 0.001s...
[t-SNE] Computed neighbors for 2737 samples in 0.160s...
[t-SNE] Computed conditional probabilities for sample 1000 / 2737
[t-SNE] Computed conditional probabilities for sample 2000 / 2737
[t-SNE] Computed conditional probabilities for sample 2737 / 2737
[t-SNE] Mean sigma: 0.163726
[t-SNE] Computed conditional probabilities in 0.064s
[t-SNE] Iteration 50: error = 75.4980774, gradient norm = 0.0654334 (50 iterations in 0.458s)
[t-SNE] Iteration 100: error = 64.2444077, gradient norm = 0.0310032 (50 iterations in 0.343s)
[t-SNE] Iteration 150: error = 61.0552635, gradient norm = 0.0197300 (50 iterations in 0.347s)
[t-SNE] Iteration 200: error = 59.4657860, gradient norm = 0.0162632 (50 iterations in 0.364s)
[t-SNE] Iteration 250: error = 58.4764214, gradient norm = 0.0149651 (50 iterations in 0.337s)
[t-SNE] KL divergence after 250 iterations with early exaggeration: 58.476421
[t-SNE] Iteration 300: error = 1.0790139, gradient norm = 0.0012137 (50 iterations in 0.350s)
[t-SNE] Iteration 350: error = 0.6867451, gradient norm = 0.0004739 (50 iterations in 0.443s)
[t-SNE] Iteration 400: error = 0.5491161, gradient norm = 0.0003004 (50 iterations in 0.343s)
[t-SNE] Iteration 450: error = 0.4869504, gradient norm = 0.0002194 (50 iterations in 0.363s)
[t-SNE] Iteration 500: error = 0.4529892, gradient norm = 0.0001802 (50 iterations in 0.366s)
[t-SNE] Iteration 550: error = 0.4323447, gradient norm = 0.0001711 (50 iterations in 0.351s)
[t-SNE] Iteration 600: error = 0.4193160, gradient norm = 0.0001389 (50 iterations in 0.336s)
[t-SNE] Iteration 650: error = 0.4087503, gradient norm = 0.0001240 (50 iterations in 0.362s)
[t-SNE] Iteration 700: error = 0.3996340, gradient norm = 0.0001202 (50 iterations in 0.351s)
[t-SNE] Iteration 750: error = 0.3921644, gradient norm = 0.0001145 (50 iterations in 0.352s)
[t-SNE] Iteration 800: error = 0.3869039, gradient norm = 0.0001125 (50 iterations in 0.381s)
[t-SNE] Iteration 850: error = 0.3825578, gradient norm = 0.0001033 (50 iterations in 0.350s)
[t-SNE] Iteration 900: error = 0.3786403, gradient norm = 0.0000984 (50 iterations in 0.335s)
[t-SNE] Iteration 950: error = 0.3752485, gradient norm = 0.0000936 (50 iterations in 0.349s)
[t-SNE] Iteration 1000: error = 0.3723453, gradient norm = 0.0000970 (50 iterations in 0.368s)
[t-SNE] KL divergence after 1000 iterations: 0.372345
```

Done..

Creating plot for this t-sne visualization..
saving this plot as image in present working directory...



Out[37]:

```
Done
1079    Sitting_right
1684    Sitting_left
732      Sitting
1185    Sitting_right
1423    Sitting_left
...
279      Sitting
2041    Standing_down
664      Sitting
1318    Sitting_left
723      Sitting
Name: class, Length: 1915, dtype: object
```

Modeling

In [38]:

```
from sklearn import preprocessing
from sklearn import tree
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn import svm
from sklearn import metrics
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import f_classif
```

In [39]:

```

kernal_evals = dict()
def evaluate_classification(model, name, X_train, X_test, y_train, y_test):
    train_accuracy = metrics.accuracy_score(y_train, model.predict(X_train))
    test_accuracy = metrics.accuracy_score(y_test, model.predict(X_test))

    train_precision = metrics.precision_score(y_train, model.predict(X_train), average='micro')
    test_precision = metrics.precision_score(y_test, model.predict(X_test), average='micro')

    train_recall = metrics.recall_score(y_train, model.predict(X_train), average='micro')
    test_recall = metrics.recall_score(y_test, model.predict(X_test), average='micro')

    kernal_evals[str(name)] = [train_accuracy, test_accuracy, train_precision, test_precision, train_recall, test_recall]
    print("Training Accuracy " + str(name) + " {} Test Accuracy ".format(train_accuracy*100) + str(name) + " {}".format(test_acc
    print("Training Precesion " + str(name) + " {} Test Precesion ".format(train_precision*100) + str(name) + " {}".format(test_
    print("Training Recall " + str(name) + " {} Test Recall ".format(train_recall*100) + str(name) + " {}".format(test_recall*10

    actual = y_test
    predicted = model.predict(X_test)
    confusion_matrix = metrics.confusion_matrix(actual, predicted)
    cm_display = metrics.ConfusionMatrixDisplay(confusion_matrix = confusion_matrix, display_labels = ['Bending', 'Sitting', 'Sit
        'Standing_down', 'Standing_up'])

    fig, ax = plt.subplots(figsize=(10,10))
    ax.grid(False)
    cm_display.plot(ax=ax)

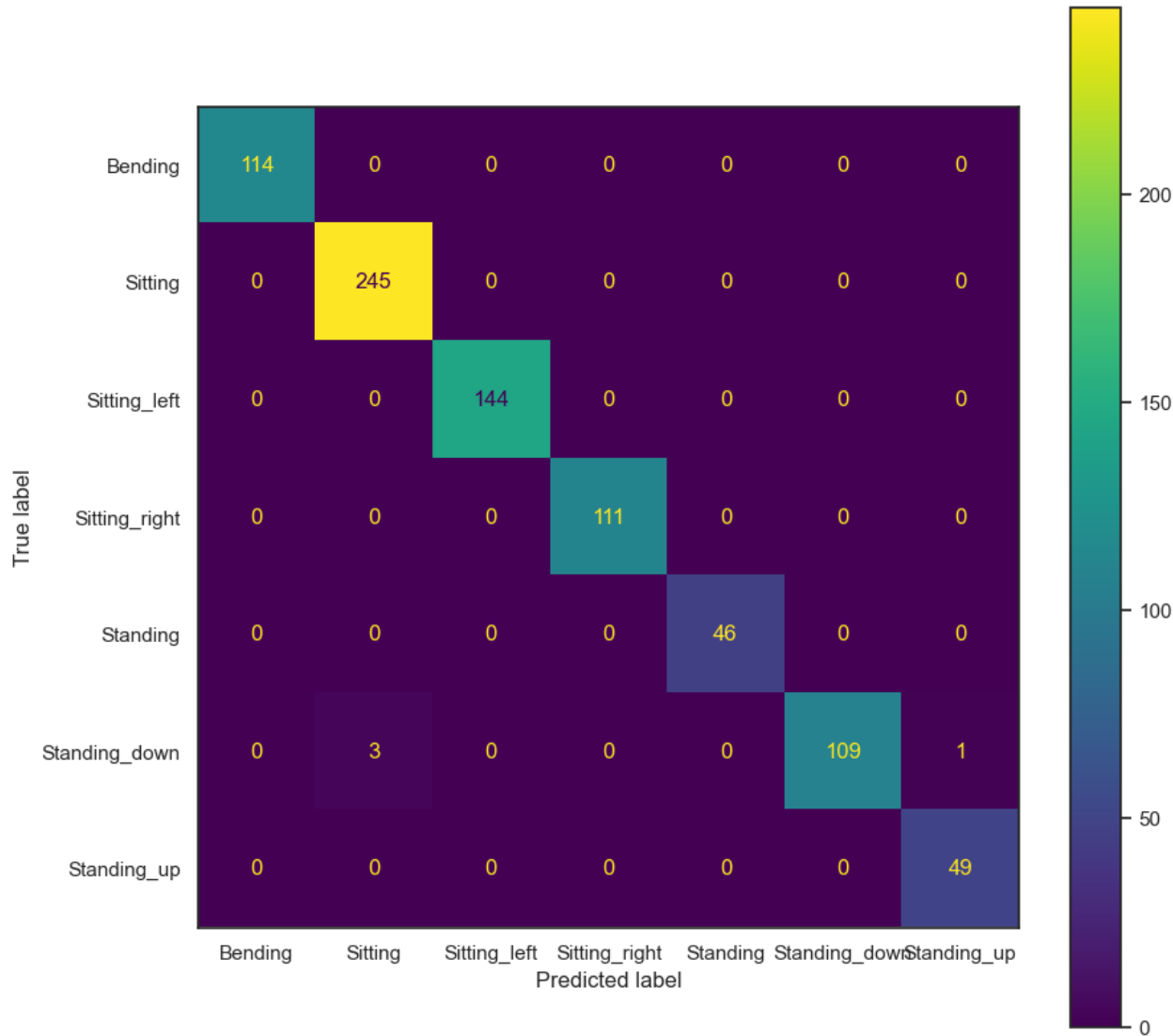
```

1.Logistic Regression

In [40]:

```
lr = LogisticRegression().fit(X_train, y_train)
evaluate_classification(lr, "Logistic Regression", X_train, X_test, y_train, y_test)
```

Training Accuracy Logistic Regression 99.94778067885117 Test Accuracy Logistic Regression 99.51338199513383
Training Precesion Logistic Regression 99.94778067885117 Test Precesion Logistic Regression 99.51338199513383
Training Recall Logistic Regression 99.94778067885117 Test Recall Logistic Regression 99.51338199513383

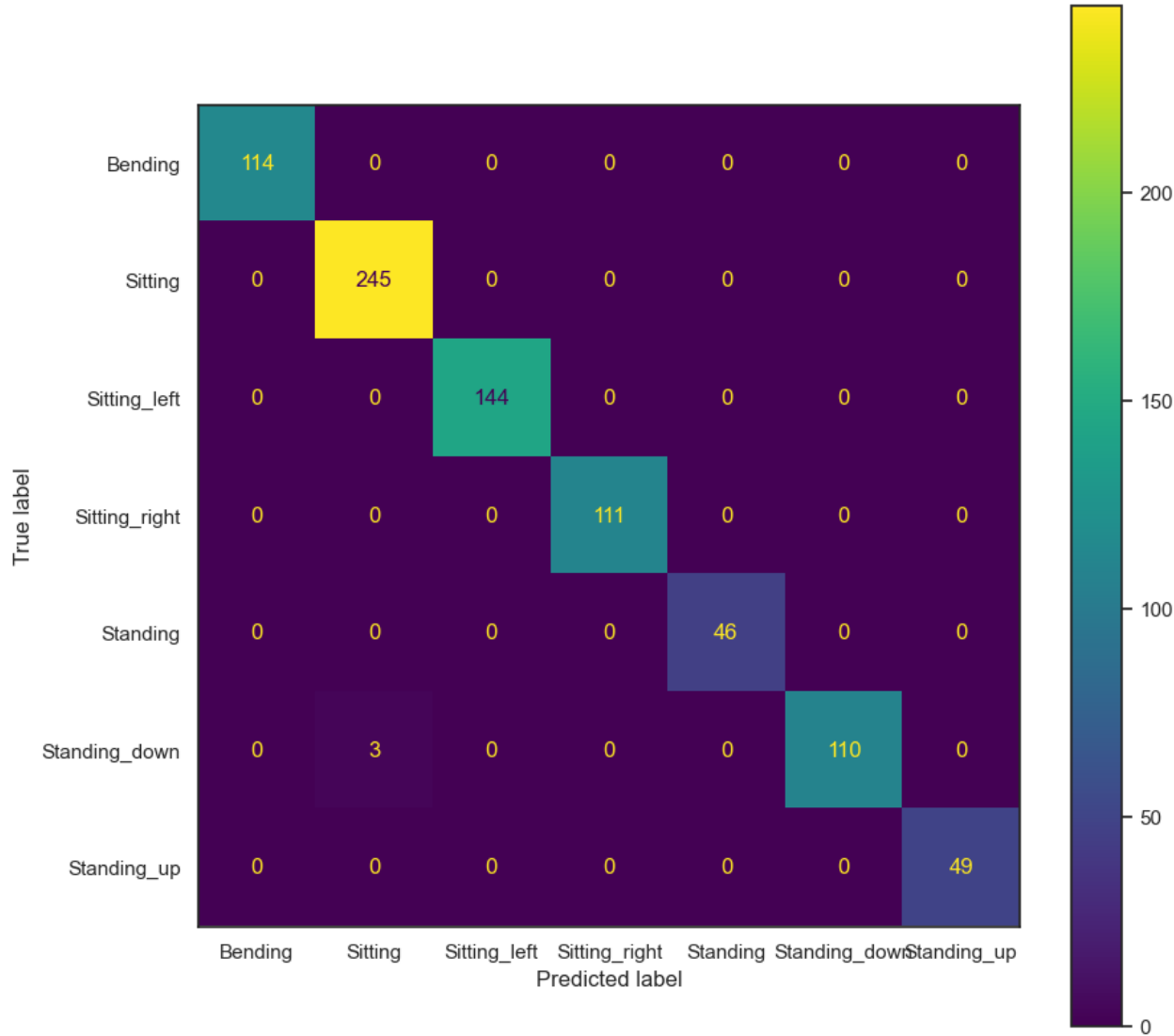


2.Linear Discriminant Analysis

In [41]:

```
lda = LinearDiscriminantAnalysis().fit(X_train, y_train)
evaluate_classification(lda, "LinearDiscriminantAnalysis", X_train, X_test, y_train, y_test)
```

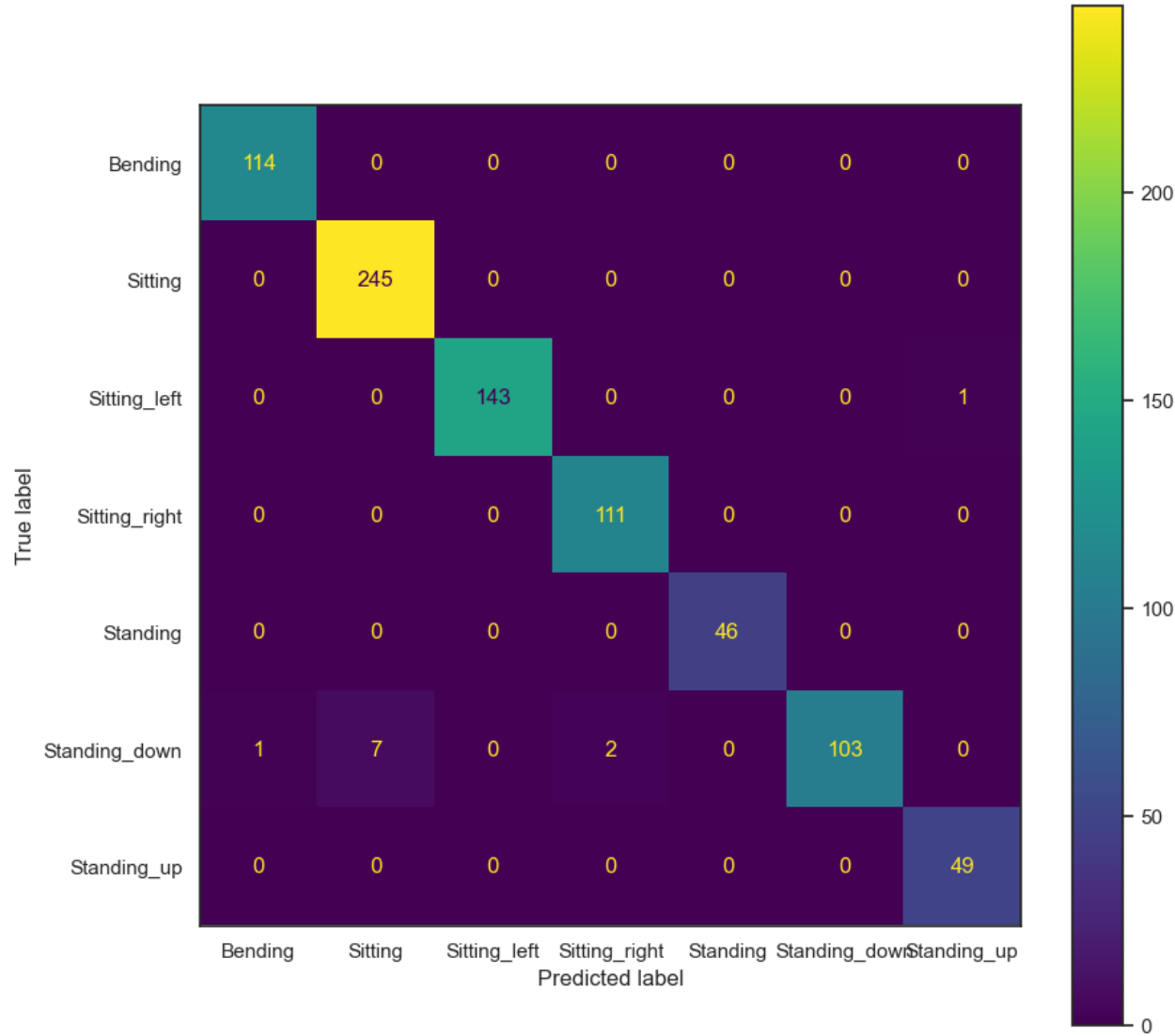
Training Accuracy LinearDiscriminantAnalysis 99.94778067885117 Test Accuracy LinearDiscriminantAnalysis 99.63503649635037
Training Precesion LinearDiscriminantAnalysis 99.94778067885117 Test Precesion LinearDiscriminantAnalysis 99.63503649635037
Training Recall LinearDiscriminantAnalysis 99.94778067885117 Test Recall LinearDiscriminantAnalysis 99.63503649635037



3.k-nearest neighbors

```
knn = KNeighborsClassifier(n_neighbors=30).fit(X_train, y_train)
evaluate_classification(knn, "KNeighborsClassifier", X_train, X_test, y_train, y_test)
```

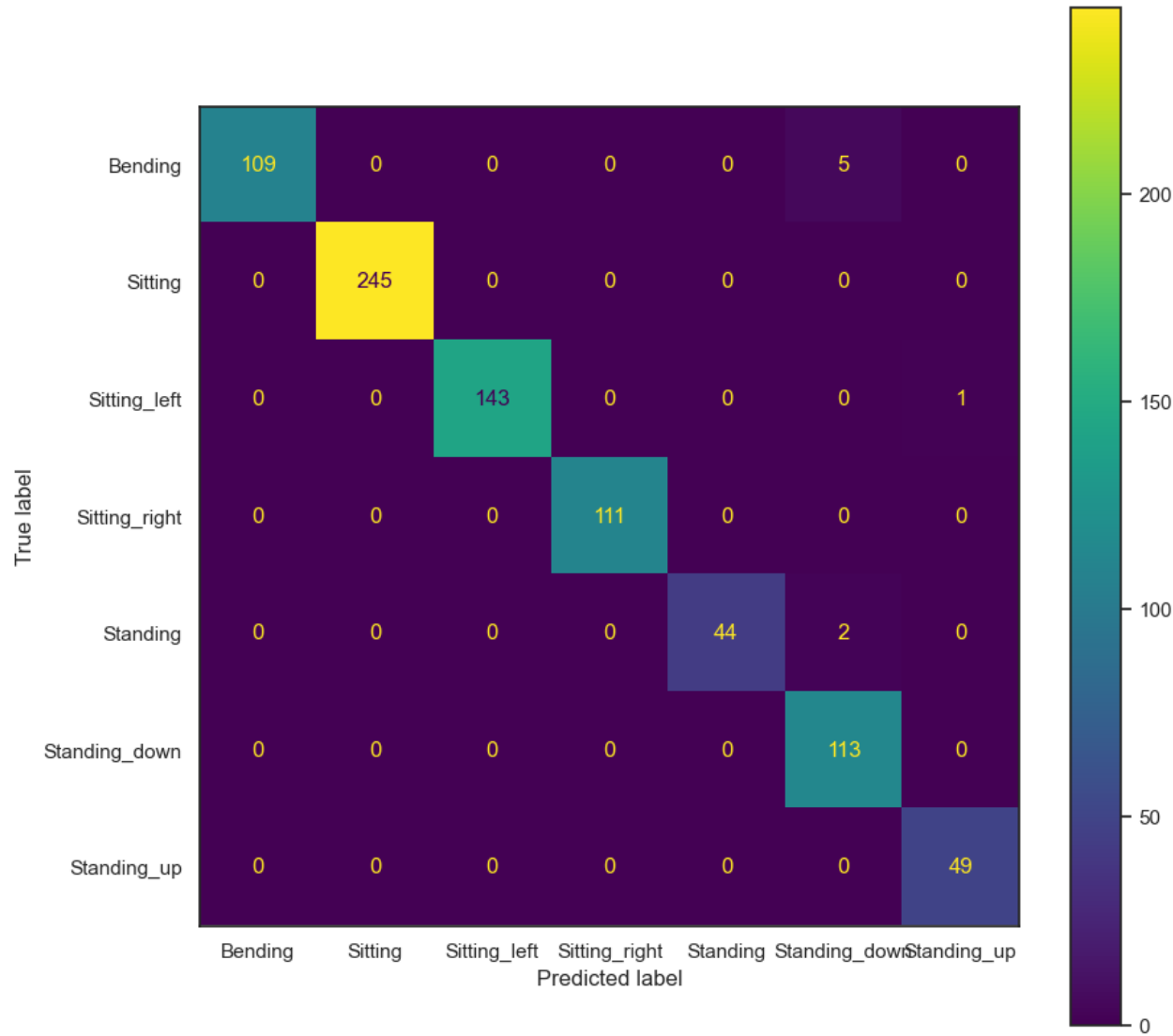
Training Accuracy KNeighborsClassifier 98.90339425587467 Test Accuracy KNeighborsClassifier 98.661800486618
Training Precesion KNeighborsClassifier 98.90339425587467 Test Precesion KNeighborsClassifier 98.661800486618
Training Recall KNeighborsClassifier 98.90339425587467 Test Recall KNeighborsClassifier 98.661800486618



4.Naive Bayes

```
gnb = GaussianNB().fit(X_train, y_train)
evaluate_classification(gnb, "GaussianNB", X_train, X_test, y_train, y_test)
```

Training Accuracy GaussianNB 98.9556135770235 Test Accuracy GaussianNB 99.02676399026764
Training Precesion GaussianNB 98.9556135770235 Test Precesion GaussianNB 99.02676399026764
Training Recall GaussianNB 98.9556135770235 Test Recall GaussianNB 99.02676399026764

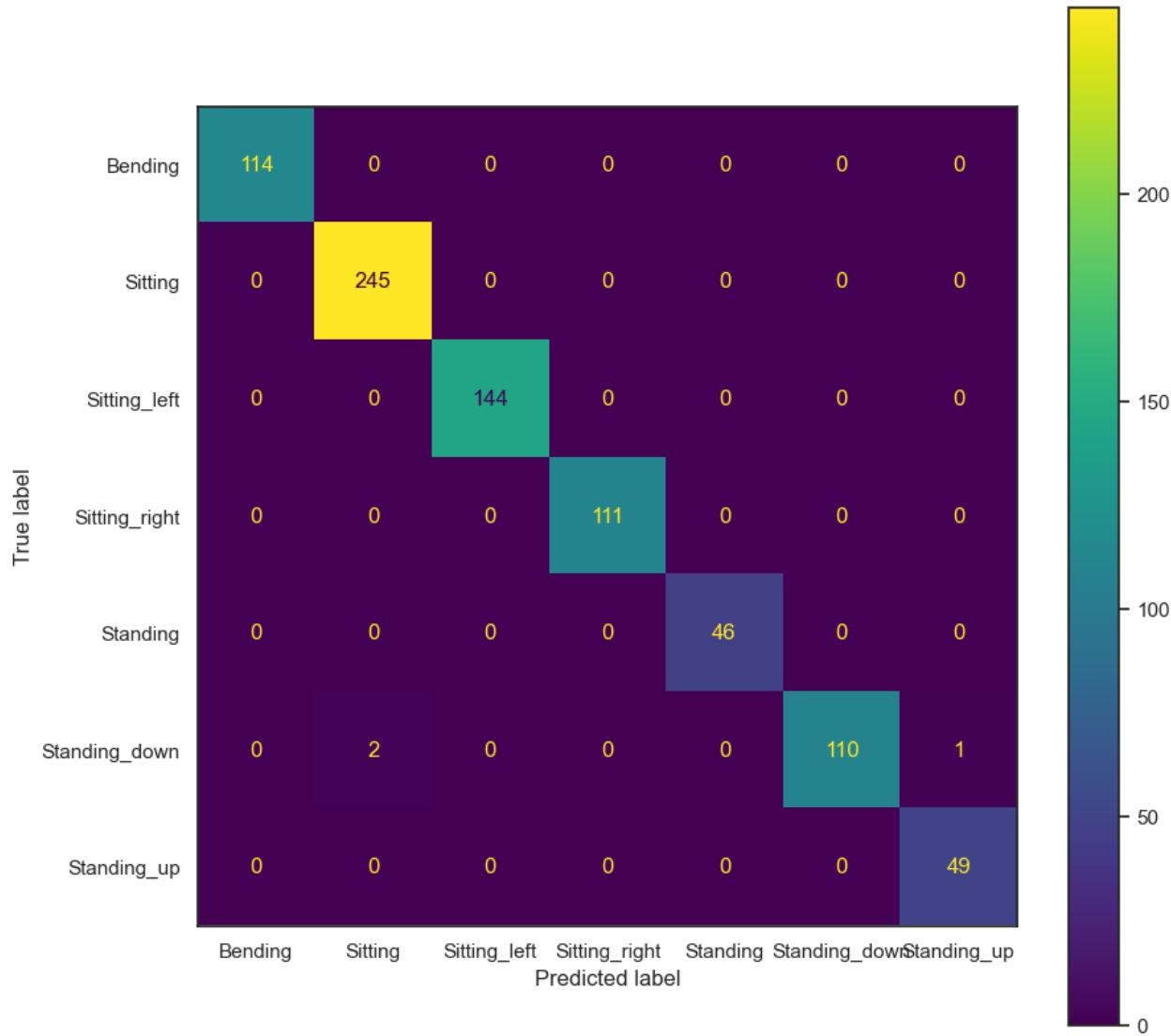


5.Support Vector Machines

In [44]:

```
lin_svc = svm.LinearSVC().fit(X_train, y_train)
evaluate_classification(lin_svc, "Linear SVC", X_train, X_test, y_train, y_test)
```

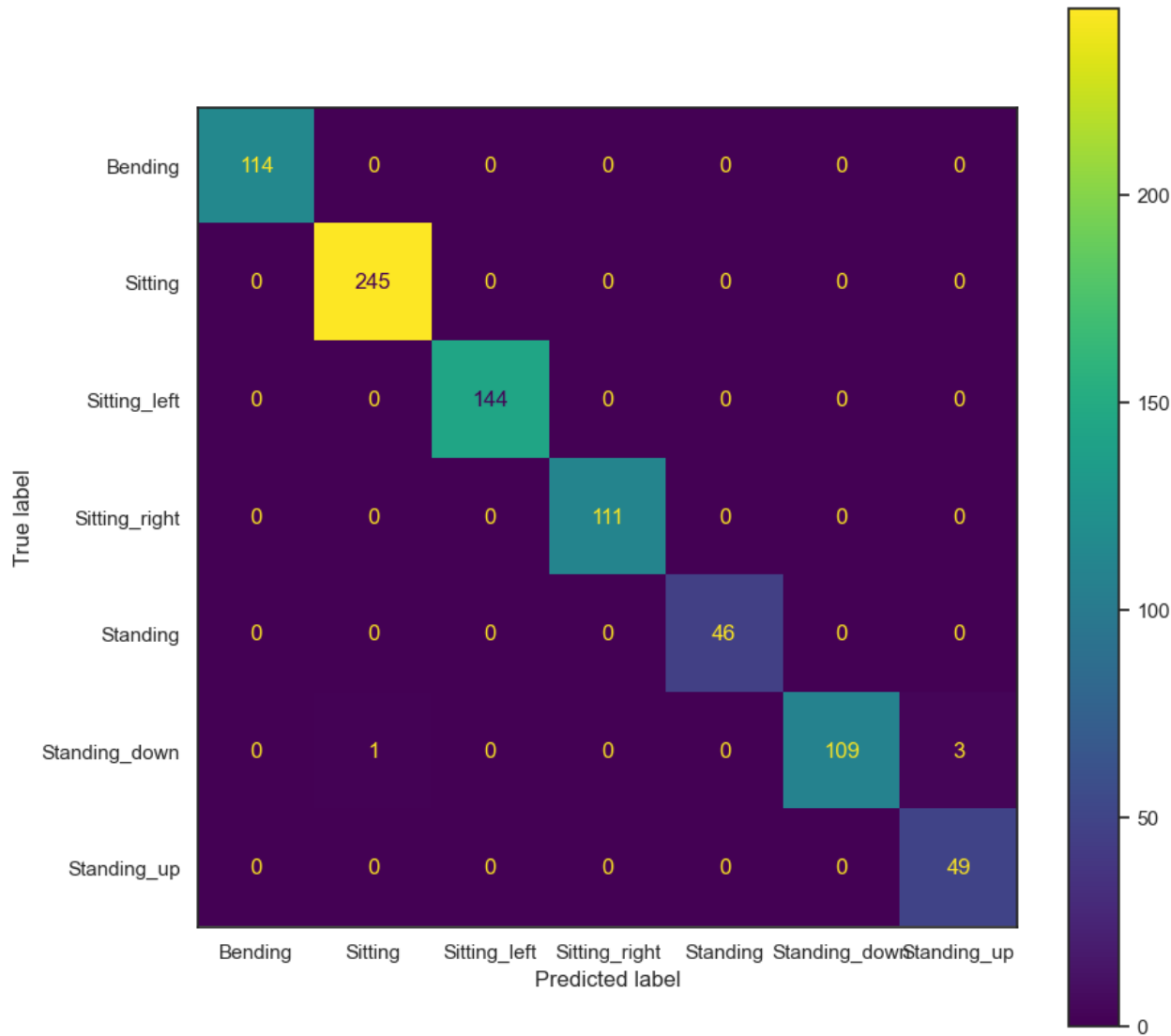
Training Accuracy Linear SVC 99.94778067885117 Test Accuracy Linear SVC 99.63503649635037
Training Precesion Linear SVC 99.94778067885117 Test Precesion Linear SVC 99.63503649635037
Training Recall Linear SVC 99.94778067885117 Test Recall Linear SVC 99.63503649635037



In [45]:

```
rbf = svm.SVC(kernel='rbf').fit(X_train, y_train)
evaluate_classification(rbf, "RBF SVC", X_train, X_test, y_train,y_test)
```

Training Accuracy RBF SVC 99.73890339425587 Test Accuracy RBF SVC 99.51338199513383
Training Precesion RBF SVC 99.73890339425587 Test Precesion RBF SVC 99.51338199513383
Training Recall RBF SVC 99.73890339425587 Test Recall RBF SVC 99.51338199513383

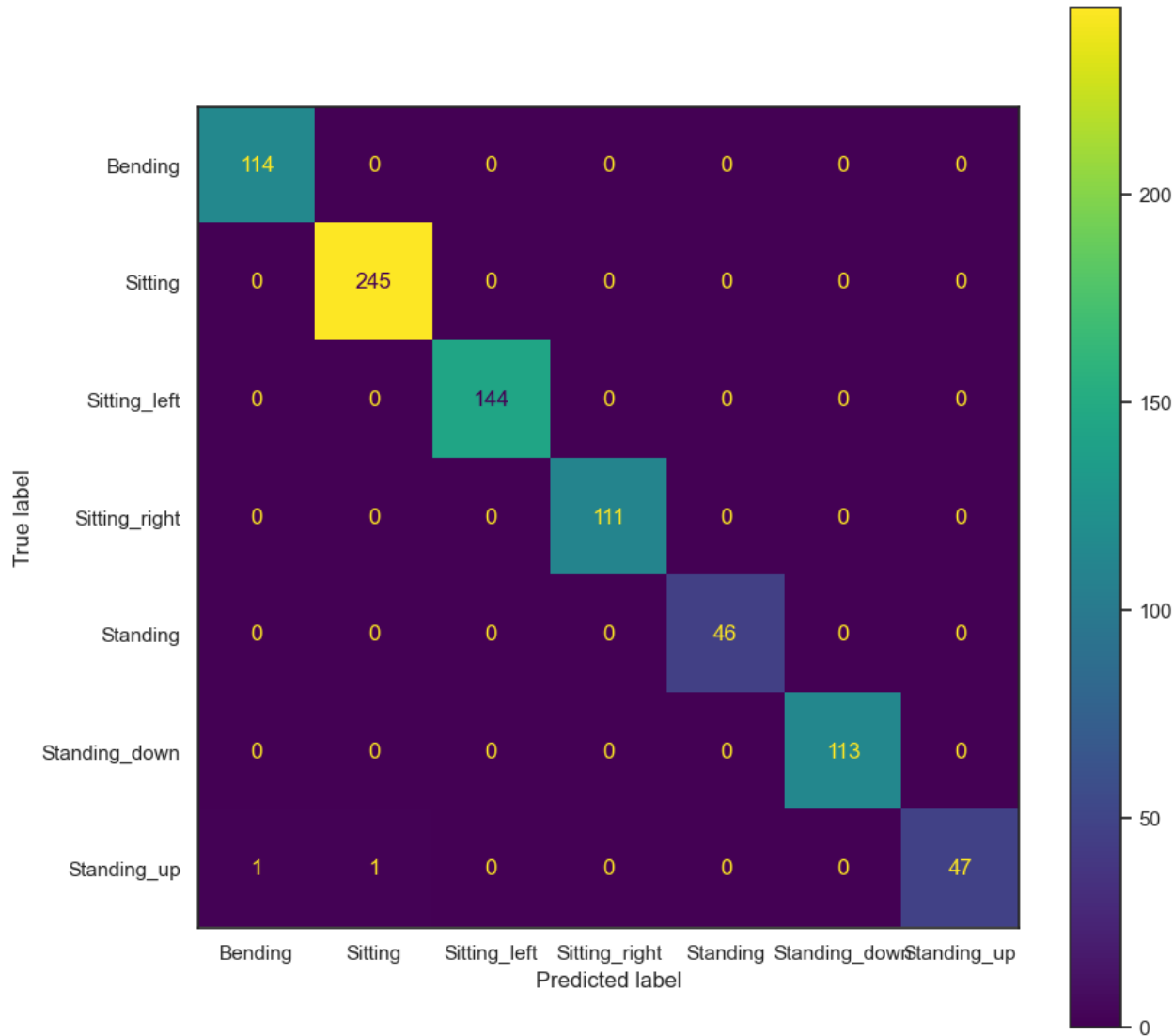


6.Decision Tree

In [46]:

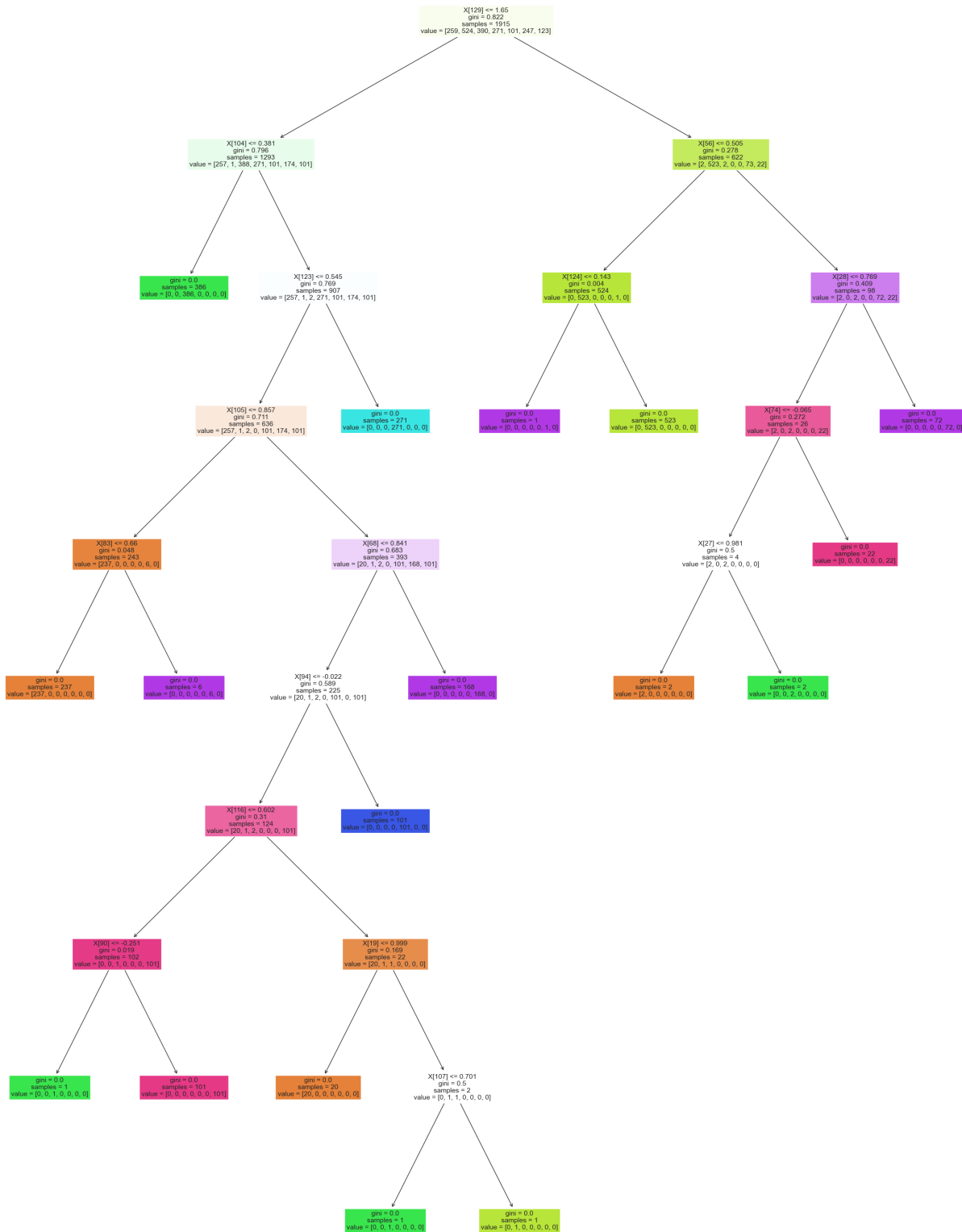
```
dt = DecisionTreeClassifier().fit(X_train, y_train)
evaluate_classification(dt, "DecisionTreeClassifier", X_train, X_test, y_train, y_test)
```

Training Accuracy DecisionTreeClassifier 100.0 Test Accuracy DecisionTreeClassifier 99.7566909975669
Training Precesion DecisionTreeClassifier 100.0 Test Precesion DecisionTreeClassifier 99.7566909975669
Training Recall DecisionTreeClassifier 100.0 Test Recall DecisionTreeClassifier 99.7566909975669



H

```
fig = plt.figure(figsize=(30,40))
tree.plot_tree(dt , filled=True)
plt.show()
```

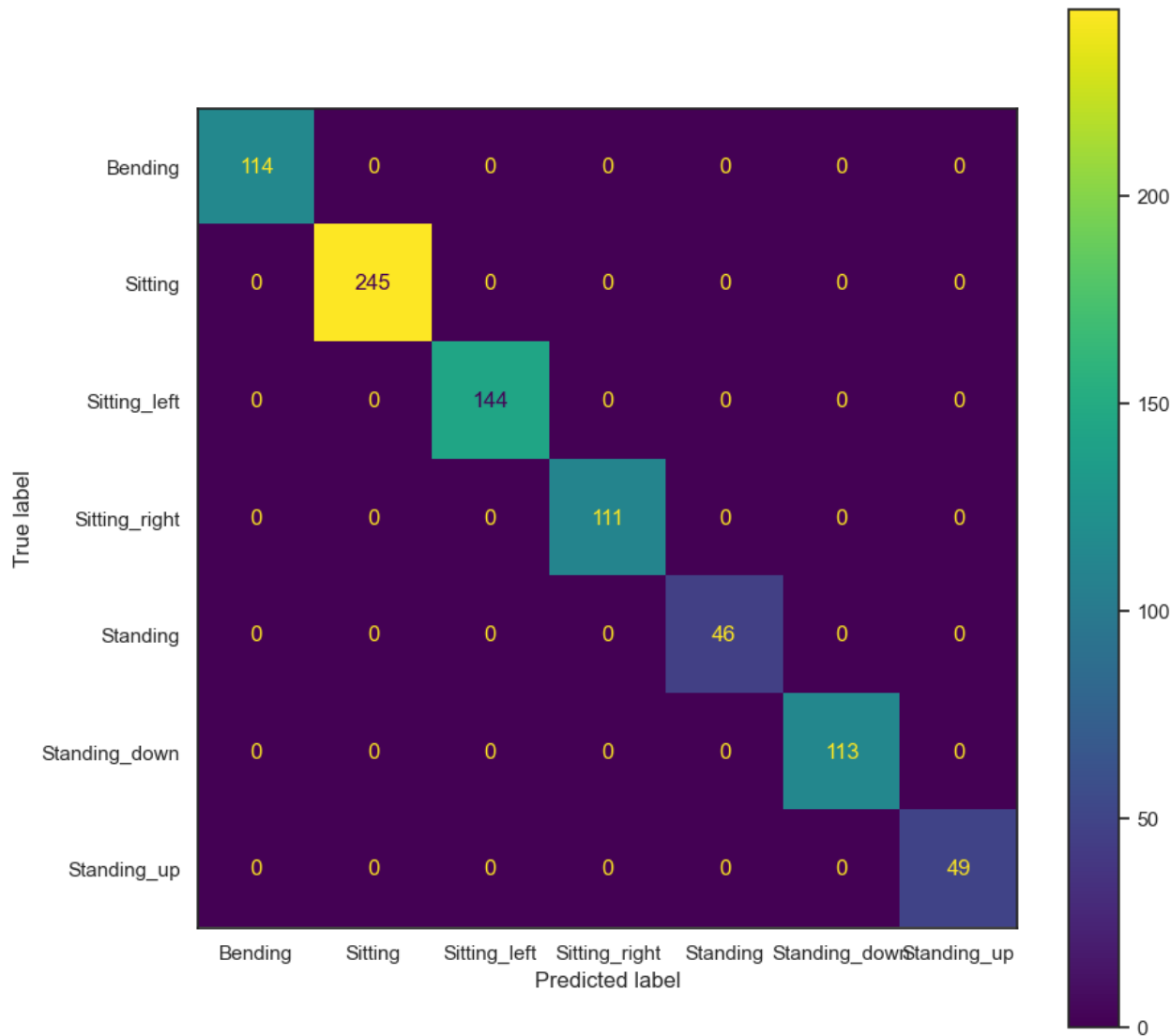


7.RandomForest Classifier

In [49]:

```
rf = RandomForestClassifier().fit(X_train, y_train)
evaluate_classification(rf, "RandomForestClassifier", X_train, X_test, y_train, y_test)
```

Training Accuracy RandomForestClassifier 100.0 Test Accuracy RandomForestClassifier 100.0
Training Precesion RandomForestClassifier 100.0 Test Precesion RandomForestClassifier 100.0
Training Recall RandomForestClassifier 100.0 Test Recall RandomForestClassifier 100.0



Overall evaluation comparison plot: Accuracy,Precesion & Recall

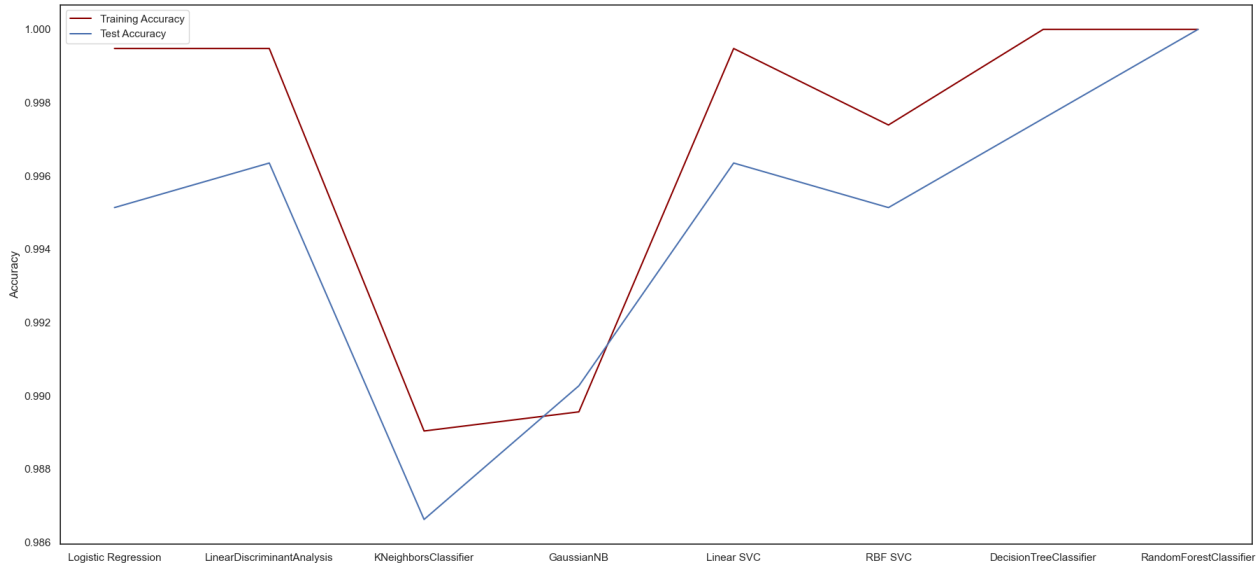
1.Accuracy

In [50]:

```

keys = [key for key in kernel_evals.keys()]
values = [value for value in kernel_evals.values()]
fig, ax = plt.subplots(figsize=(22, 10))
ax.plot(np.arange(len(keys)), [value[0] for value in values], color='darkred')
ax.plot(np.arange(len(keys)), [value[1] for value in values], color='b')
ax.legend(["Training Accuracy", "Test Accuracy"])
ax.set_xticklabels(keys)
ax.set_xticks(np.arange(len(keys)))
plt.ylabel("Accuracy")
plt.show()

```



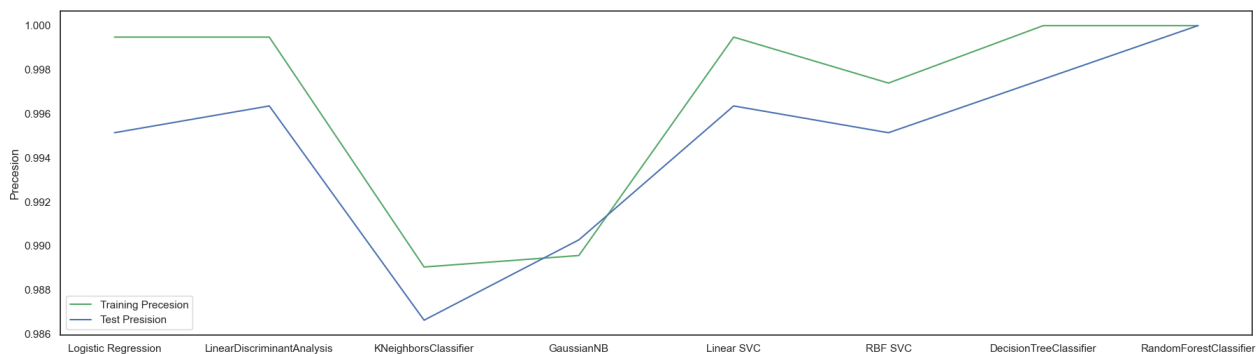
2.Precision

In [51]:

```

keys = [key for key in kernel_evals.keys()]
values = [value for value in kernel_evals.values()]
fig, ax = plt.subplots(figsize=(22, 6))
ax.plot(np.arange(len(keys)), [value[2] for value in values], color='g')
ax.plot(np.arange(len(keys)), [value[3] for value in values], color='b')
ax.legend(["Training Precision", "Test Precision"])
ax.set_xticklabels(keys)
ax.set_xticks(np.arange(len(keys)))
plt.ylabel("Precision")
plt.show()

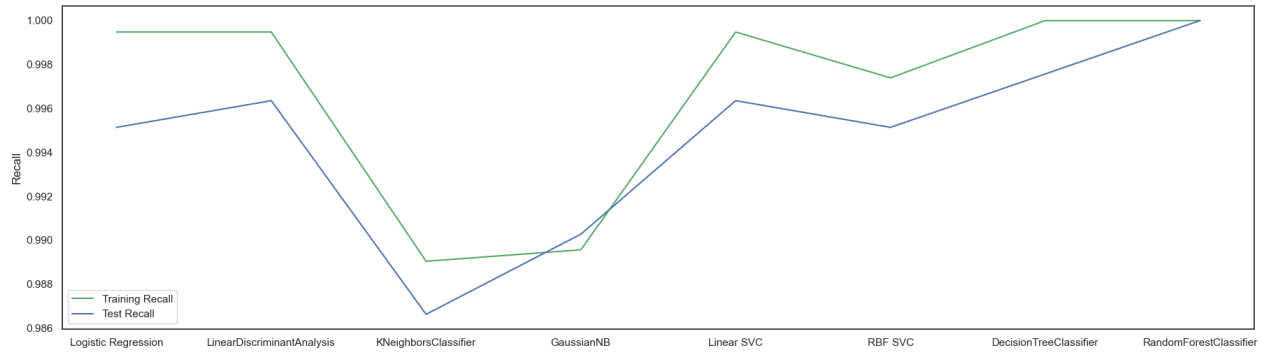
```



3.Recall

In [52]:

```
keys = [key for key in kernal_evals.keys()]
values = [value for value in kernal_evals.values()]
fig, ax = plt.subplots(figsize=(22, 6))
ax.plot(np.arange(len(keys)), [value[4] for value in values], color='g')
ax.plot(np.arange(len(keys)), [value[5] for value in values], color='b')
ax.legend(["Training Recall", "Test Recall"])
ax.set_xticklabels(keys)
ax.set_xticks(np.arange(len(keys)))
plt.ylabel("Recall")
plt.show()
```



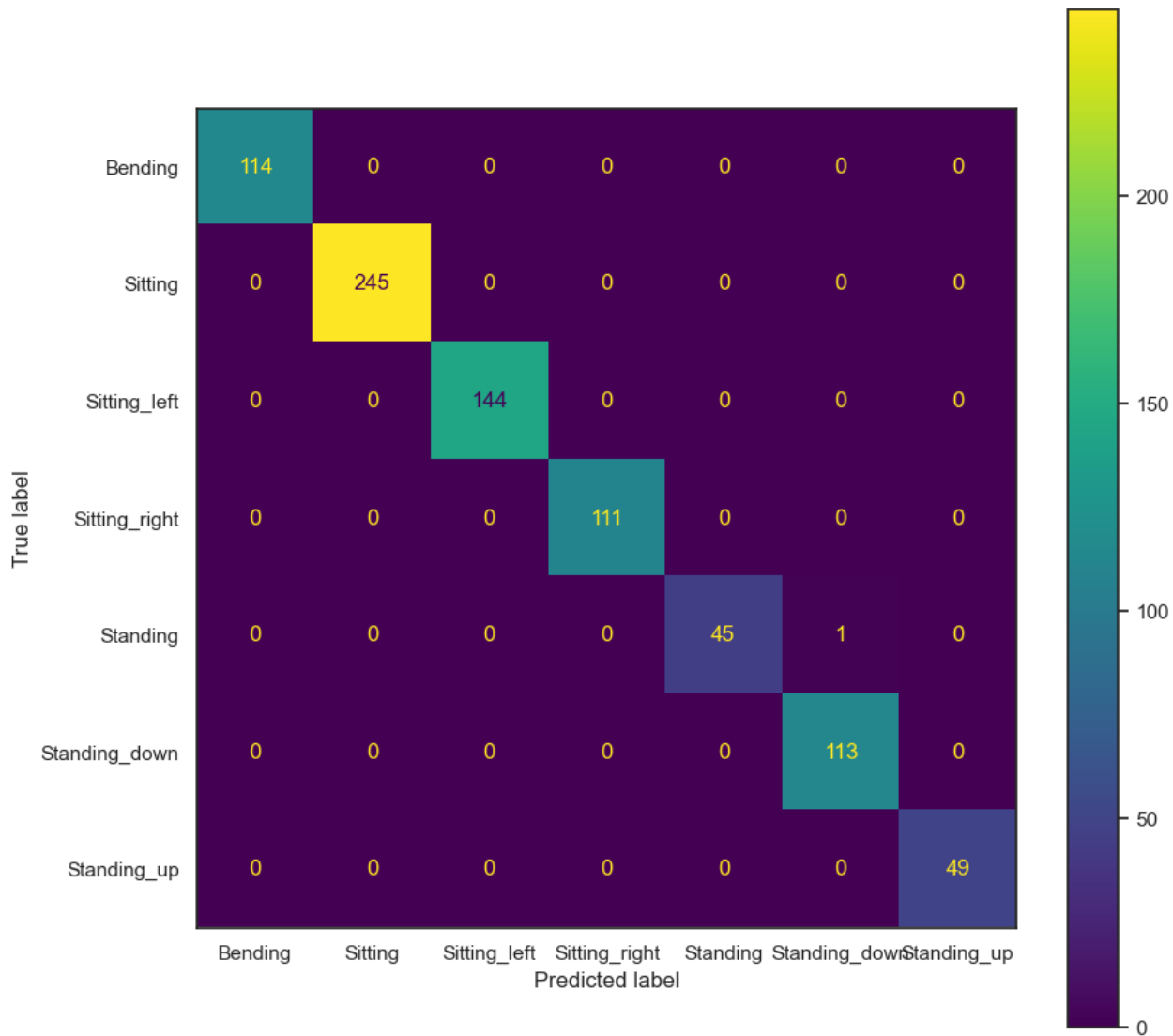
From the evaluation comparison: RandomForestClassifier model is the best

Feature reduction: PCA

In [53]:

```
rf_pca = RandomForestClassifier().fit(X_train_pca, y_train_pca)
evaluate_classification(rf_pca, "RandomForestClassifier(pca)", X_train_pca, X_test_pca, y_train_pca, y_test_pca)
```

Training Accuracy RandomForestClassifier(pca) 100.0 Test Accuracy RandomForestClassifier(pca) 99.87834549878345
 Training Precesion RandomForestClassifier(pca) 100.0 Test Precesion RandomForestClassifier(pca) 99.87834549878345
 Training Recall RandomForestClassifier(pca) 100.0 Test Recall RandomForestClassifier(pca) 99.87834549878345



Deep learning model

In [54]:

```
# import required Liabraries for Nueral Network
import keras
import tensorflow as tf
from keras import layers
from keras.callbacks import EarlyStopping
from tensorflow.keras import utils
from tensorflow.keras.initializers import RandomNormal
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Activation
```

In [55]:

```
# Label_encoder object knows how to understand word Labels
label_encoder = preprocessing.LabelEncoder()
```

In [56]:

```
def Graph_Drawing(history, title):
    x = np.arange(len(history.history["val_accuracy"]))

    plt.plot(x, history.history["loss"], label="loss")
    plt.plot(x, history.history["val_loss"], label="val_loss")

    plt.plot(x, history.history["accuracy"], label="accuracy")
    plt.plot(x, history.history["val_accuracy"], label="val_accuracy")

    plt.title(title)

    plt.legend()

    plt.show()
```

MODEL training

In [80]:

```
def MLP_Model(data):
    model = keras.Sequential([
        layers.Input(shape=df.shape[1:]),

        layers.Dense(1000, activation="relu"),
        #     Layers.Dropout(0.1),
        #     Layers.BatchNormalization(),

        layers.Dense(500, activation="relu"),
        #     Layers.Dropout(0.25),
        #     Layers.BatchNormalization(),

        layers.Dense(6, activation="sigmoid"),
    ], name="MLP_Sequential")

    model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])

    print(model.summary())

    return model
```

In [83]:

```
train_features = df.drop(["class"], axis=1)
test_features = df.drop(["class"], axis=1)

train_labels = df["class"].replace(['Bending', 'Sitting', 'Sitting_left', 'Sitting_right', 'Standing',
    'Standing_down', 'Standing_up' ], [0, 1, 2, 3, 4, 5, 6])
test_labels = df["class"].replace(['Bending', 'Sitting', 'Sitting_left', 'Sitting_right', 'Standing',
    'Standing_down', 'Standing_up' ], [0, 1, 2, 3, 4, 5, 6])
print(train_features.shape)
print(test_features.shape)

print(train_labels.shape)
print(test_labels.shape)
```

```
(2737, 132)
(2737, 132)
(2737,)
(2737,)
```

Multi Layered Perceptron

In [99]:

```
#Multi Layered Perceptron
mlp_model = MLP_Model(train_features)
mlp_model.summary()
```

Model: "MLP_Sequential"

Layer (type)	Output Shape	Param #
dense_12 (Dense)	(None, 1000)	134000
dense_13 (Dense)	(None, 500)	500500
dense_14 (Dense)	(None, 6)	3006

=====
 Total params: 637,506
 Trainable params: 637,506
 Non-trainable params: 0

None
Model: "MLP_Sequential"

Layer (type)	Output Shape	Param #
dense_12 (Dense)	(None, 1000)	134000
dense_13 (Dense)	(None, 500)	500500
dense_14 (Dense)	(None, 6)	3006

=====
 Total params: 637,506
 Trainable params: 637,506
 Non-trainable params: 0

In []:

In [100]:

```
def CNN_Model(data):
    model = keras.Sequential(name="CNN_Sequential")
    print(data.shape[1:])
    model.add( layers.Input(shape=data.shape[1:]) )

    model.add(layers.Flatten())

    model.add( layers.Dense(64, activation="relu", name="dense_1") )
    model.add( layers.Dense(128, activation="relu", name="dense_2") )
    model.add( layers.Dense(6, activation="softmax", name="output") )
    model.compile(optimizer=tf.keras.optimizers.SGD(),
                  loss="sparse_categorical_crossentropy", metrics=["accuracy"])

    print(model.summary())

    return model
```

In [101]:

```
#Convolutional Neural Network
cnn_model = CNN_Model(train_features)
cnn_model.summary()
```

(132,)
Model: "CNN_Sequential"

Layer (type)	Output Shape	Param #
flatten_4 (Flatten)	(None, 132)	0
dense_1 (Dense)	(None, 64)	8512
dense_2 (Dense)	(None, 128)	8320
output (Dense)	(None, 6)	774

=====

Total params: 17,606
Trainable params: 17,606
Non-trainable params: 0

None
Model: "CNN_Sequential"

Layer (type)	Output Shape	Param #
flatten_4 (Flatten)	(None, 132)	0
dense_1 (Dense)	(None, 64)	8512
dense_2 (Dense)	(None, 128)	8320
output (Dense)	(None, 6)	774

=====

Total params: 17,606
Trainable params: 17,606
Non-trainable params: 0

In [98]:

```
earlystopping = EarlyStopping(patience=30, monitor="val_accuracy", restore_best_weights=True)
```

In [102]:

```
test_features.shape
```

Out[102]:
(2737, 132)

In [103]:

```

cnn_predict = cnn_model.predict( test_features )
original = test_labels

cnn_predict = np.argmax(cnn_predict, axis=1)
cf = metrics.confusion_matrix(cnn_predict, original)

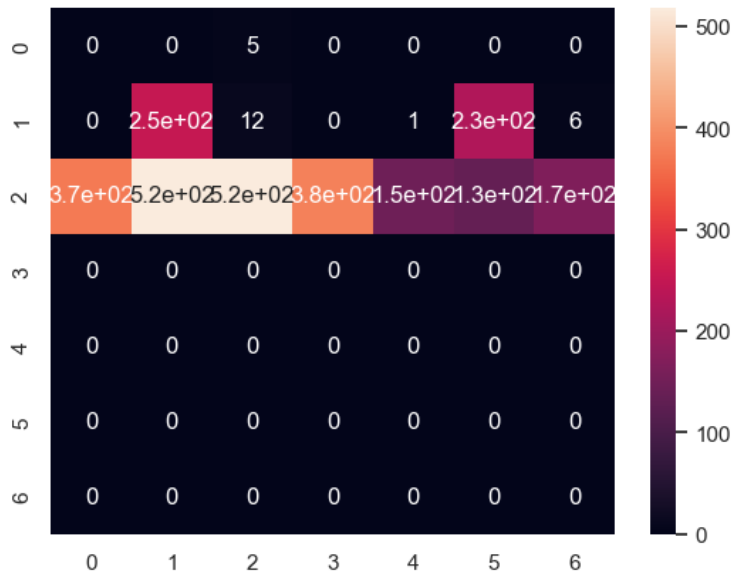
sns.heatmap(cf, annot=True)

```

86/86 [=====] - 0s 2ms/step

Out[103]:

<AxesSubplot:>



In [112]:

```

def CNN_LSTM_Model(data):
    model = keras.Sequential(name="CNN_Sequential")
    print(data.shape[1:])
    model.add( layers.Input(shape=data.shape[1:]) )
    model.add( layers.Dense(64, activation="relu", name="dense_1") )
    model.add( layers.Dense(128, activation="relu", name="dense_2") )
    model.add( layers.Dense(6, activation="softmax", name="output") )

    model.compile(optimizer=tf.keras.optimizers.SGD(),
                  loss="sparse_categorical_crossentropy", metrics=["accuracy"])

    print(model.summary())

    return model

```

In [113]:

```
#Convolutional Neural Network + Long Short Term Memory Model
cnn_lstm_model = CNN_LSTM_Model(train_features)
cnn_lstm_model.summary()
```

(132,)
Model: "CNN_Sequential"

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 64)	8512
dense_2 (Dense)	(None, 128)	8320
output (Dense)	(None, 6)	774

=====
Total params: 17,606
Trainable params: 17,606
Non-trainable params: 0

None
Model: "CNN_Sequential"

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 64)	8512
dense_2 (Dense)	(None, 128)	8320
output (Dense)	(None, 6)	774

=====
Total params: 17,606
Trainable params: 17,606
Non-trainable params: 0

In [115]:

```
cnn_lstm_predict = cnn_lstm_model.predict( test_features )
original = test_labels

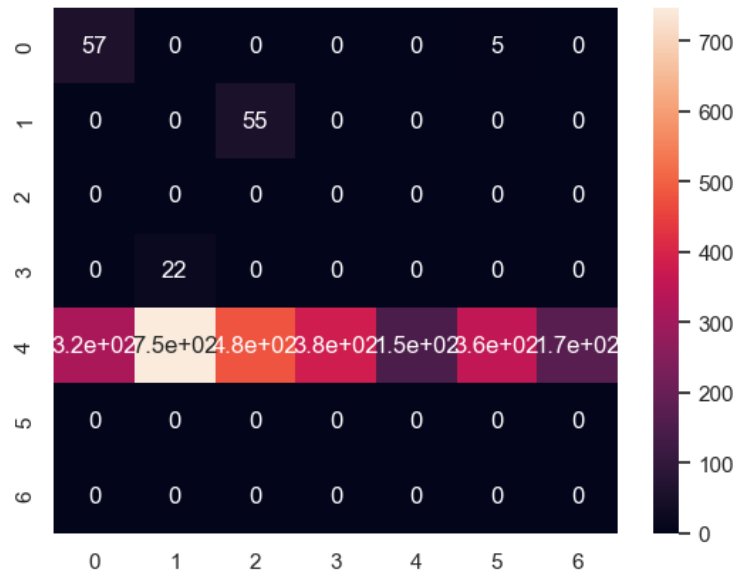
cnn_lstm_predict = np.argmax(cnn_lstm_predict, axis=1)
cf = metrics.confusion_matrix(cnn_lstm_predict, original)

sns.heatmap(cf, annot=True)
```

86/86 [=====] - 0s 1ms/step

Out[115]:

<AxesSubplot:>



In [117]:

```
#Training Models (Selected Features)
```

In [118]:

```
train_features = df.drop(["class"], axis=1)  
test_features = df.drop(["class"], axis=1)
```

In [119]:

```
features = pd.concat([train_features, test_features], axis=0, ignore_index=False)  
labels = pd.concat([train_labels, test_labels], axis=0, ignore_index=False)  
  
features = PCA(n_components=36).fit_transform(features, labels)  
  
train_features, test_features, train_labels, test_labels = train_test_split(features, labels, train_size=0.8, random_state=2)
```

In [120]:

```
train_features.shape
```

Out[120]:

```
(4379, 36)
```

In [121]:

```
test_features.shape
```

Out[121]:

```
(1095, 36)
```

In [122]:

```
train_labels.shape
```

Out[122]:

```
(4379,)
```

In [123]:

```
test_labels.shape
```

Out[123]:

```
(1095,)
```

In [126]:

```
#Multi Layered Perceptron
mlp_model = MLP_Model(train_features)
mlp_model.summary()

earlystopping = EarlyStopping( monitor="val_accuracy", restore_best_weights=True)
```

Model: "MLP_Sequential"

Layer (type)	Output Shape	Param #
=====		
dense_18 (Dense)	(None, 1000)	134000
dense_19 (Dense)	(None, 500)	500500
dense_20 (Dense)	(None, 6)	3006
=====		
Total params: 637,506		
Trainable params: 637,506		
Non-trainable params: 0		

None
Model: "MLP_Sequential"

Layer (type)	Output Shape	Param #
=====		
dense_18 (Dense)	(None, 1000)	134000
dense_19 (Dense)	(None, 500)	500500
dense_20 (Dense)	(None, 6)	3006
=====		
Total params: 637,506		
Trainable params: 637,506		
Non-trainable params: 0		

In [131]:

```
#Convolutional Neural Network
cnn_model = CNN_Model(train_features)
cnn_model.summary()
```

(36,)
Model: "CNN_Sequential"

Layer (type)	Output Shape	Param #
=====		
flatten_5 (Flatten)	(None, 36)	0
dense_1 (Dense)	(None, 64)	2368
dense_2 (Dense)	(None, 128)	8320
output (Dense)	(None, 6)	774
=====		
Total params: 11,462		
Trainable params: 11,462		
Non-trainable params: 0		

None
Model: "CNN_Sequential"

Layer (type)	Output Shape	Param #
=====		
flatten_5 (Flatten)	(None, 36)	0
dense_1 (Dense)	(None, 64)	2368
dense_2 (Dense)	(None, 128)	8320
output (Dense)	(None, 6)	774
=====		
Total params: 11,462		
Trainable params: 11,462		
Non-trainable params: 0		

In [132]:

```
test_features.shape
```

Out[132]:

(1095, 36)

In [133]:

```
cnn_predict = cnn_model.predict( test_features )
original = test_labels

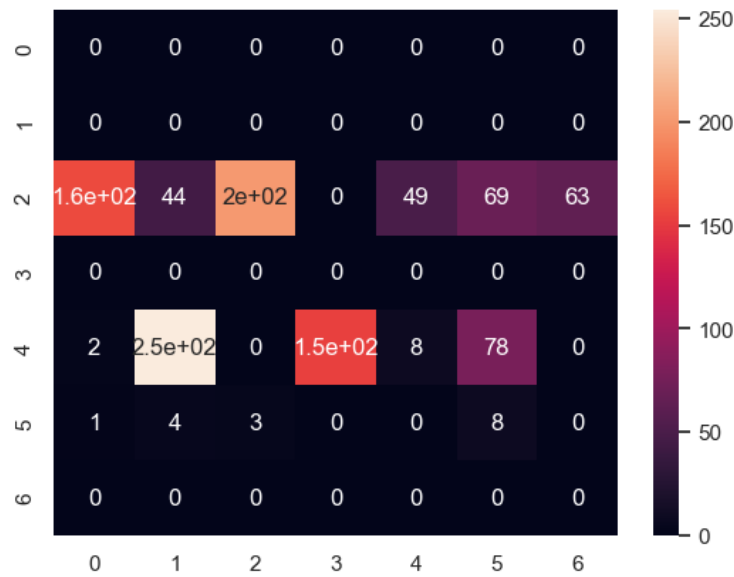
cnn_predict = np.argmax(cnn_predict, axis=1)
cf = metrics.confusion_matrix(cnn_predict, original)

sns.heatmap(cf, annot=True)
```

35/35 [=====] - 0s 1ms/step

Out[133]:

<AxesSubplot:>



In [135]:

```
#Convolutional Neural Network + Long Short Term Memory Model
cnn_lstm_model = CNN_LSTM_Model(train_features)
cnn_lstm_model.summary()
```

(36,)

Model: "CNN_Sequential"

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 64)	2368
dense_2 (Dense)	(None, 128)	8320
output (Dense)	(None, 6)	774

=====

Total params: 11,462
Trainable params: 11,462
Non-trainable params: 0

None

Model: "CNN_Sequential"

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 64)	2368
dense_2 (Dense)	(None, 128)	8320
output (Dense)	(None, 6)	774

=====

Total params: 11,462
Trainable params: 11,462
Non-trainable params: 0

In [136]:

```
cnn_lstm_predict = cnn_lstm_model.predict( test_features )
original = test_labels

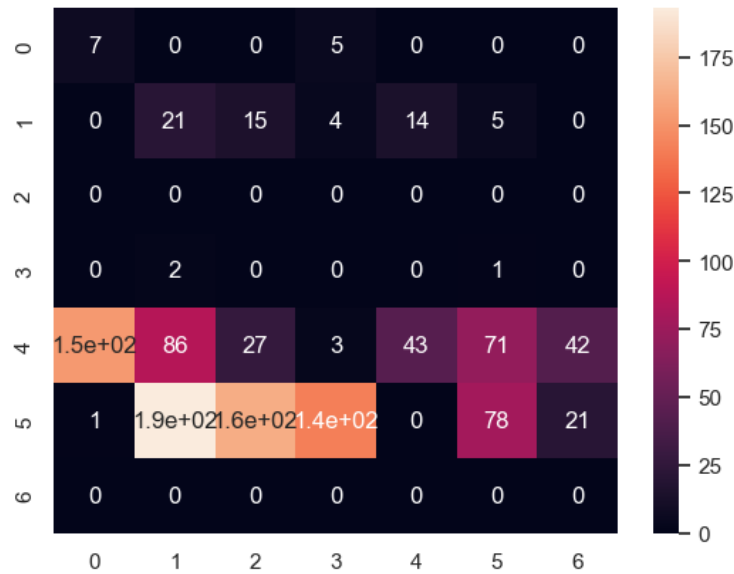
cnn_lstm_predict = np.argmax(cnn_lstm_predict, axis=1)
cf = metrics.confusion_matrix(cnn_lstm_predict, original)

sns.heatmap(cf, annot=True)
```

35/35 [=====] - 0s 1ms/step

Out[136]:

<AxesSubplot:>



Train Machine Learning Classification Model

In []: ⌵

In []: ⌵

In []: ⌵

In []: ⌵

In []: ⌵

In []: ⌵

In []: ⌵

In []: ⌵

In []: ⌵

In []: ⌵

In []: ⌵

In []: ⌵

In []: ⌵

In []: ⌵

In []: ⌵

In [173]: ⌵

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression, RidgeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
```

In [174]:

```
pipelines = {  
    'lr':make_pipeline(StandardScaler(), LogisticRegression()),  
    'rc':make_pipeline(StandardScaler(), RidgeClassifier()),  
    'rf':make_pipeline(StandardScaler(), RandomForestClassifier()),  
    'gb':make_pipeline(StandardScaler(), GradientBoostingClassifier()),  
}
```

In [175]:

```
fit_models = {}  
for algo, pipeline in pipelines.items():  
    model = pipeline.fit(X_train, y_train)  
    fit_models[algo] = model
```

In [176]:

```
fit_models
```

Out[176]:

```
{'lr': Pipeline(steps=[('standardscaler', StandardScaler()),  
                        ('logisticregression', LogisticRegression())]),  
 'rc': Pipeline(steps=[('standardscaler', StandardScaler()),  
                        ('ridgeclassifier', RidgeClassifier())]),  
 'rf': Pipeline(steps=[('standardscaler', StandardScaler()),  
                        ('randomforestclassifier', RandomForestClassifier())]),  
 'gb': Pipeline(steps=[('standardscaler', StandardScaler()),  
                        ('gradientboostingclassifier', GradientBoostingClassifier())])}
```

In [177]:



```
fit_models['rc'].predict(X_test)
```

Out[177]:

```

array(['Sitting_right', 'Sitting_left', 'Sitting_left', 'Standing_up',
      'Sitting', 'Bending', 'Standing_up', 'Sitting', 'Bending',
      'Standing', 'Standing_down', 'Sitting', 'Standing_up', 'Sitting',
      'Sitting_left', 'Bending', 'Sitting', 'Standing_down', 'Sitting',
      'Standing', 'Sitting', 'Sitting', 'Sitting_left', 'Sitting',
      'Standing_down', 'Standing_down', 'Standing', 'Bending', 'Sitting',
      'Bending', 'Standing_down', 'Sitting', 'Bending', 'Standing_down',
      'Sitting', 'Sitting_left', 'Bending', 'Standing_down', 'Sitting',
      'Sitting_left', 'Sitting', 'Standing_down', 'Bending', 'Sitting',
      'Sitting_left', 'Bending', 'Bending', 'Sitting', 'Sitting',
      'Standing_down', 'Standing_up', 'Sitting', 'Sitting',
      'Sitting_right', 'Sitting_left', 'Bending', 'Sitting', 'Bending',
      'Sitting', 'Sitting_right', 'Sitting', 'Sitting', 'Sitting_right',
      'Sitting', 'Sitting_right', 'Sitting', 'Sitting', 'Sitting',
      'Standing_down', 'Bending', 'Sitting', 'Sitting_left',
      'Sitting_right', 'Sitting', 'Sitting', 'Sitting', 'Standing_down',
      'Sitting', 'Sitting', 'Sitting_left', 'Standing_down', 'Bending',
      'Bending', 'Bending', 'Sitting', 'Sitting', 'Standing_down',
      'Standing_down', 'Standing_down', 'Standing_down', 'Sitting_left',
      'Bending', 'Sitting', 'Sitting_right', 'Standing_down', 'Bending',
      'Sitting_left', 'Standing_down', 'Sitting_right', 'Sitting_left',
      'Standing_up', 'Sitting', 'Sitting_right', 'Sitting_left',
      'Sitting_right', 'Sitting_left', 'Sitting_left', 'Sitting_left',
      'Sitting', 'Standing', 'Sitting', 'Standing_down', 'Standing_down',
      'Sitting', 'Standing_up', 'Sitting', 'Standing_up', 'Bending',
      'Sitting_right', 'Standing', 'Standing_up', 'Sitting', 'Standing',
      'Sitting_right', 'Sitting', 'Sitting', 'Sitting_left', 'Sitting',
      'Sitting_left', 'Sitting', 'Standing', 'Standing_down',
      'Sitting_right', 'Sitting', 'Sitting', 'Sitting', 'Sitting',
      'Sitting_left', 'Sitting_left', 'Standing_down', 'Standing_down',
      'Sitting', 'Sitting_left', 'Sitting', 'Bending', 'Sitting',
      'Sitting', 'Sitting_left', 'Sitting', 'Bending', 'Sitting',
      'Sitting_left', 'Sitting', 'Standing_down', 'Sitting', 'Bending',
      'Sitting_left', 'Bending', 'Sitting', 'Sitting', 'Standing',
      'Standing_down', 'Standing_up', 'Sitting_left', 'Sitting',
      'Standing', 'Sitting_left', 'Sitting_right', 'Sitting_left',
      'Sitting', 'Sitting_right', 'Sitting_left', 'Sitting',
      'Standing_down', 'Sitting_left', 'Sitting', 'Bending',
      'Sitting_right', 'Sitting', 'Sitting', 'Bending', 'Sitting_left',
      'Sitting', 'Standing_up', 'Standing', 'Bending', 'Sitting',
      'Bending', 'Sitting_left', 'Standing_down', 'Bending',
      'Standing_up', 'Standing_down', 'Bending', 'Sitting_left',
      'Sitting', 'Sitting_left', 'Sitting', 'Sitting_left',
      'Sitting_right', 'Bending', 'Standing_down', 'Standing', 'Sitting',
      'Sitting', 'Sitting_right', 'Bending', 'Bending', 'Sitting',
      'Standing', 'Standing_down', 'Sitting_left', 'Sitting',
      'Standing_up', 'Sitting_left', 'Sitting_right', 'Bending',
      'Sitting', 'Standing_down', 'Sitting_right', 'Standing',
      'Standing', 'Sitting', 'Sitting_left', 'Sitting_right',
      'Standing_down', 'Standing_down', 'Bending', 'Sitting_left',
      'Standing_down', 'Standing_down', 'Sitting_right', 'Bending',
      'Sitting', 'Standing_up', 'Bending', 'Sitting', 'Sitting',
      'Sitting', 'Standing_down', 'Sitting_right', 'Bending',
      'Sitting_left', 'Standing_down', 'Standing_down', 'Sitting_right',
      'Sitting', 'Sitting', 'Standing', 'Sitting', 'Sitting_right',
      'Sitting_left', 'Standing', 'Sitting', 'Sitting_left', 'Sitting',
      'Standing_down', 'Standing_up', 'Sitting_left', 'Sitting',
      'Standing_down', 'Sitting', 'Bending', 'Standing_up', 'Sitting',
      'Bending', 'Standing_down', 'Sitting_right', 'Sitting_right',
      'Standing_down', 'Sitting_left', 'Standing_up', 'Standing_up',
      'Standing_up', 'Sitting_left', 'Sitting_left', 'Sitting',
      'Sitting', 'Sitting_left', 'Standing_down', 'Standing_down',
      'Sitting_left', 'Sitting_right', 'Sitting', 'Sitting_left',
      'Sitting_right', 'Sitting', 'Sitting', 'Sitting_left', 'Sitting',
      'Sitting', 'Sitting', 'Sitting_left', 'Sitting_right',
      'Sitting_left', 'Sitting_right', 'Standing_down', 'Standing_up',
      'Bending', 'Standing', 'Standing_up', 'Standing_down', 'Sitting',
      'Sitting', 'Sitting_right', 'Standing_up', 'Sitting', 'Sitting',
      'Standing_down', 'Sitting_right', 'Sitting_left', 'Sitting',
      'Sitting', 'Bending', 'Sitting_left', 'Sitting', 'Bending',
      'Sitting_left', 'Sitting_left', 'Sitting_left', 'Sitting',
      'Sitting_right', 'Standing', 'Sitting_left', 'Sitting', 'Sitting',
      'Sitting_left', 'Sitting_right', 'Sitting_right', 'Sitting',
      'Bending', 'Sitting_left', 'Standing_down', 'Sitting_left',
      'Sitting', 'Sitting_right', 'Sitting_left', 'Sitting_right',
      'Standing_down', 'Sitting_left', 'Sitting_right', 'Sitting_left',
      'Sitting_left', 'Sitting_left', 'Sitting', 'Bending',
      'Sitting_left', 'Bending', 'Sitting', 'Standing_down', 'Standing',
      'Sitting_right', 'Standing_up', 'Sitting_right', 'Standing_up',
      'Standing_up', 'Standing_up', 'Sitting_left', 'Sitting',
      'Sitting_left', 'Bending', 'Sitting_right', 'Bending', 'Sitting',

```

Evaluate and Serialize Model

```

5134149878345
rc 0.995109513389nding', 'Standing_down', 'Standing', 'Sitting',
rf 1.0'Sitting', 'Sitting_left', 'Sitting_left', 'Standing_down',
gb 0.9987834149878345,'Standing', 'Standing_down', 'Sitting_left',
'Sitting_left', 'Standing_down', 'Sitting_right', 'Sitting_left',
'Bending', 'Sitting', 'Standing_down', 'Sitting', 'Sitting',
'Standing_down', 'Standing_down', 'Sitting', 'Bending',
'Sitting_right', 'Sitting_right', 'Sitting', 'Sitting', 'Sitting',
'Sitting', 'Bending', 'Standing_down', 'Sitting_left',
'Sitting_left', 'Standing_up', 'Sitting_left', 'Standing',
'Sitting', 'Sitting_left', 'Sitting_right', 'Sitting',
'Standing_down', 'Bending', 'Sitting', 'Bending', 'Bending',
'Sitting_right', 'Sitting', 'Sitting_right', 'Standing_down',
'Bending', 'Sitting_right', 'Sitting', 'Standing_up',
'Sitting_right', 'Sitting_right', 'Bending', 'Sitting',
'Sitting_left', 'Standing_up', 'Sitting_left', 'Sitting_left',
'Standing', 'Standing_down', 'Sitting_right', 'Sitting',
'Standing', 'Standing_down', 'Bending', 'Sitting', 'Bending',
'Sitting', 'Standing_down', 'Sitting_right', 'Sitting', 'Sitting',
'Sitting', 'Sitting_left', 'Sitting', 'Standing', 'Bending',
'Bending', 'Standing_down', 'Sitting', 'Standing_down',
'Standing_down', 'Bending', 'Standing', 'Sitting', 'Sitting',
'Standing_down', 'Standing', 'Sitting', 'Standing_down', 'Sitting',
'Standing_up', 'Sitting_left', 'Sitting_left', 'Sitting_right',
'Sitting', 'Standing_down', 'Sitting_left', 'Sitting_left',
'Sitting_left', 'Standing_down', 'Bending', 'Sitting_right',
'Bending', 'Sitting', 'Standing_down', 'Standing', 'Sitting_left',
'Sitting_right', 'Sitting_right', 'Sitting', 'Standing',
'Sitting_right', 'Sitting', 'Standing', 'Sitting_left',
'Standing_up', 'Sitting_right', 'Sitting', 'Sitting', 'Sitting',
'Standing_down', 'Sitting_left', 'Bending', 'Sitting_right',
'Sitting_left', 'Sitting_right', 'Bending', 'Standing_down',
'Bending', 'Sitting_right', 'Sitting', 'Sitting', 'Sitting_right',
'Standing_up', 'Sitting_left', 'Standing_up', 'Bending',
'Standing_down', 'Sitting', 'Sitting', 'Sitting', 'Sitting_right',
'Sitting_left', 'Sitting', 'Sitting', 'Sitting', 'Standing_down',
'Standing', 'Sitting_right', 'Sitting', 'Bending', 'Sitting',
'Sitting', 'Sitting_right', 'Sitting', 'Sitting_left',
'Sitting_right', 'Standing_down', 'Sitting_right', 'Sitting',
'Standing_down', 'Sitting_right', 'Sitting_right', 'Bending',
'Standing_down', 'Sitting_left', 'Bending', 'Sitting',
'Standing_down', 'Sitting_left', 'Bending', 'Standing_up',
'Sitting_left', 'Sitting_right', 'Bending', 'Sitting',
'Standing_up', 'Standing', 'Bending', 'Sitting', 'Bending',
'Sitting', 'Sitting_left', 'Standing', 'Sitting', 'Sitting',
'Sitting', 'Sitting', 'Bending', 'Sitting', 'Sitting',
'Standing_up', 'Sitting_right', 'Sitting', 'Sitting',
'Sitting_right', 'Sitting', 'Sitting', 'Sitting_left', 'Standing',
'Sitting_left', 'Sitting_left', 'Sitting', 'Bending', 'Sitting',
'Sitting_right', 'Sitting', 'Sitting', 'Sitting_right', 'Sitting',
'Sitting_left', 'Standing_up', 'Bending', 'Sitting', 'Sitting',
'Sitting_left', 'Sitting_right', 'Sitting_right', 'Standing_down',
'Sitting_left', 'Bending', 'Sitting', 'Sitting_left',
'Sitting_left', 'Sitting_right', 'Sitting_left', 'Bending',
'Sitting_right', 'Sitting', 'Sitting_right', 'Sitting', 'Sitting',
'Bending', 'Sitting_right', 'Standing', 'Sitting_left',

```

```
In [180]: 'Standing_up', 'Standing_down', 'Sitting_right', 'Sitting',  
fit_models['rf'].predict(X_test)  
'Standing_up', 'Standing_down', 'Standing_up', 'Sitting',  
'Bending', 'Bending', 'Bending', 'Bending', 'Standing_down',  
'Sitting_left', 'Sitting_right', 'Sitting_left', 'Sitting',  
'Standing_down', 'Sitting', 'Sitting', 'Standing_down', 'Standing',  
'Bending', 'Sitting', 'Sitting', 'Standing_down', 'Sitting',  
'Bending', 'Bending', 'Sitting', 'Standing', 'Sitting_left',  
'Sitting', 'Sitting_left', 'Sitting_left', 'Standing_up',  
'Standing_down', 'Sitting', 'Sitting_left', 'Sitting', 'Bending',  
'Sitting_right', 'Sitting', 'Bending', 'Sitting', 'Sitting_left',  
'Sitting_left', 'Bending', 'Sitting_left', 'Sitting_left',  
'Sitting', 'Standing_down', 'Bending'], dtype='<U13')
```

Out[180]:

```
array(['Sitting_right', 'Sitting_left', 'Sitting_left', 'Standing_up',  
      'Sitting', 'Bending', 'Standing_up', 'Sitting', 'Bending',  
      'Standing', 'Standing_down', 'Sitting', 'Standing_up', 'Sitting',  
      'Sitting_left', 'Bending', 'Sitting', 'Standing_down', 'Sitting',  
      'Standing', 'Sitting', 'Sitting', 'Sitting_left', 'Sitting',  
      'Standing_down', 'Standing_down', 'Standing', 'Bending', 'Sitting',  
      'Bending', 'Standing_down', 'Sitting', 'Bending', 'Standing_down',  
      'Sitting', 'Sitting_left', 'Bending', 'Standing_down', 'Sitting',  
      'Sitting_left', 'Sitting', 'Standing_down', 'Bending', 'Sitting',  
      'Sitting_left', 'Bending', 'Bending', 'Sitting', 'Sitting',  
      'Standing_down', 'Standing_up', 'Sitting', 'Sitting',  
      'Sitting_right', 'Sitting_left', 'Bending', 'Sitting', 'Bending',  
      'Sitting', 'Sitting_right', 'Sitting', 'Sitting', 'Sitting_right',  
      'Sitting', 'Sitting_right', 'Sitting', 'Sitting', 'Sitting',  
      'Standing_down', 'Bending', 'Sitting', 'Sitting_left',  
      'Sitting_right', 'Sitting', 'Sitting', 'Sitting', 'Standing_down',  
      'Sitting', 'Sitting', 'Sitting_left', 'Standing_down', 'Bending',  
      'Bending', 'Bending', 'Sitting', 'Standing_down', 'Standing_down',  
      'Standing_down', 'Standing_down', 'Standing_down', 'Sitting_left',  
      'Bending', 'Sitting', 'Sitting_right', 'Standing_down', 'Bending',  
      'Sitting_left', 'Standing_down', 'Sitting_right', 'Sitting_left',  
      'Standing_up', 'Sitting', 'Sitting_right', 'Sitting_left',  
      'Sitting_right', 'Sitting_left', 'Sitting_left', 'Sitting_left',  
      'Sitting', 'Standing', 'Sitting', 'Standing_down', 'Standing_down',  
      'Sitting', 'Standing_up', 'Sitting', 'Standing_up', 'Bending',  
      'Sitting_right', 'Standing', 'Standing_up', 'Sitting', 'Standing',  
      'Sitting_right', 'Sitting', 'Sitting', 'Sitting_left', 'Sitting',  
      'Sitting_left', 'Sitting', 'Standing', 'Standing_down',  
      'Sitting_right', 'Sitting', 'Sitting', 'Sitting', 'Sitting',  
      'Sitting_left', 'Sitting_left', 'Standing_down', 'Standing_down',  
      'Sitting', 'Sitting_left', 'Sitting', 'Bending', 'Standing_down',  
      'Sitting', 'Sitting_left', 'Sitting', 'Bending', 'Sitting',  
      'Sitting_left', 'Sitting', 'Standing_down', 'Sitting', 'Bending',  
      'Sitting_left', 'Bending', 'Sitting', 'Sitting', 'Standing',  
      'Standing_down', 'Standing_up', 'Sitting_left', 'Sitting',  
      'Standing', 'Sitting_left', 'Sitting_right', 'Sitting_left',  
      'Sitting', 'Sitting_right', 'Sitting_left', 'Sitting',  
      'Standing_down', 'Sitting_left', 'Sitting', 'Bending',  
      'Sitting_right', 'Sitting', 'Sitting', 'Bending', 'Sitting_left',  
      'Sitting', 'Standing_up', 'Standing', 'Bending', 'Sitting',  
      'Bending', 'Sitting_left', 'Standing_down', 'Bending',  
      'Standing_up', 'Standing_down', 'Bending', 'Sitting_left',  
      'Sitting', 'Sitting_left', 'Sitting', 'Sitting_left',  
      'Sitting_right', 'Bending', 'Standing_down', 'Standing', 'Sitting',  
      'Sitting_right', 'Bending', 'Standing_down', 'Standing',  
      'Standing', 'Standing_down', 'Sitting_left', 'Sitting',  
      'Standing_up', 'Sitting_left', 'Sitting_right', 'Bending',  
      'Sitting', 'Standing_down', 'Sitting_right', 'Standing',  
      'Standing', 'Sitting', 'Sitting_left', 'Sitting_right',  
      'Standing_down', 'Standing_down', 'Bending', 'Sitting_left',  
      'Standing_down', 'Standing_down', 'Sitting_right', 'Bending',  
      'Sitting', 'Standing_down', 'Bending', 'Sitting', 'Sitting',  
      'Sitting', 'Standing_down', 'Sitting_right', 'Bending',  
      'Sitting_left', 'Standing_down', 'Standing_down', 'Sitting_right',  
      'Sitting', 'Sitting', 'Standing', 'Sitting', 'Sitting_right',  
      'Sitting_left', 'Standing', 'Sitting', 'Sitting_left', 'Sitting',  
      'Standing_down', 'Standing_up', 'Sitting_left', 'Sitting',  
      'Standing_down', 'Sitting', 'Sitting', 'Bending', 'Standing_up', 'Sitting',  
      'Bending', 'Standing_down', 'Sitting_right', 'Sitting_right',  
      'Standing_down', 'Sitting_left', 'Standing_up', 'Standing_up',  
      'Standing_up', 'Sitting_left', 'Sitting_left', 'Sitting',  
      'Sitting', 'Sitting_left', 'Standing_down', 'Standing_down',  
      'Sitting_left', 'Sitting_right', 'Sitting', 'Sitting_left',  
      'Sitting_right', 'Sitting', 'Sitting', 'Sitting_left', 'Sitting',  
      'Sitting', 'Sitting', 'Sitting_left', 'Sitting_right',  
      'Sitting_left', 'Sitting_right', 'Standing_down', 'Standing_up',  
      'Bending', 'Standing', 'Standing_up', 'Standing_down', 'Sitting',  
      'Sitting', 'Sitting_right', 'Standing_up', 'Sitting', 'Sitting',  
      'Standing_down', 'Sitting_right', 'Sitting_left', 'Sitting',  
      'Sitting', 'Bending', 'Sitting_left', 'Sitting', 'Bending',  
      'Sitting_left', 'Sitting_left', 'Sitting_left', 'Sitting',  
      'Sitting_right', 'Standing', 'Sitting_left', 'Sitting', 'Sitting',  
      'Sitting_left', 'Sitting_right', 'Sitting_right', 'Sitting',  
      'Bending', 'Sitting_left', 'Standing_down', 'Sitting_left',  
      'Sitting', 'Sitting_right', 'Sitting_left', 'Sitting_right',  
      'Standing_down', 'Sitting_left', 'Sitting_right', 'Sitting_left',  
      'Sitting_left', 'Sitting_left', 'Sitting', 'Bending',  
      'Sitting_left', 'Bending', 'Sitting', 'Standing_down', 'Standing',  
      'Sitting_right', 'Standing_up', 'Sitting_right', 'Standing_up',  
      'Standing_up', 'Standing_up', 'Sitting_left', 'Sitting',  
      'Sitting_left', 'Bending', 'Sitting_right', 'Bending', 'Sitting',
```

```

'Standing_down', 'Sitting_right', 'Sitting_right', 'Sitting',
'Standing', 'Sitting', 'Standing_up', 'Sitting', 'Standing_down',
'Sitting', 'Sitting_right', 'Bending', 'Bending', 'Standing',
'Sitting', 'Sitting', 'Standing_up', 'Bending', 'Sitting_right',
'Standing_down', 'Standing_down', 'Sitting_left', 'Sitting_right',
'Sitting', 'Standing_down', 'Sitting', 'Sitting', 'Standing_down',
'Sitting', 'Sitting', 'Sitting', 'Standing_down', 'Bending',
'Standing_down', 'Sitting_left', 'Bending', 'Sitting_left',
'Sitting_left', 'Sitting_left', 'Sitting', 'Sitting', 'Sitting',
'Sitting', 'Sitting', 'Sitting_right', 'Bending', 'Sitting_right',
'Standing_up', 'Standing_down', 'Sitting_right', 'Standing_down',
'Standing_down', 'Sitting_right', 'Standing_up', 'Sitting_left',
'Sitting', 'Sitting_left', 'Standing_up', 'Sitting_left',
'Sitting_right', 'Sitting_right', 'Sitting', 'Sitting', 'Sitting',
'Sitting', 'Sitting', 'Standing_up', 'Sitting_right', 'Bending',
In [181]: Sitting_left', 'Standing', 'Bending', 'Sitting', 'Standing',
with open('body_language.pkl', 'wb') as f:
    pickle.dump(fitting, f)
    pickle.dump(fitting, f)
    Sitting_right', 'Sitting', 'Standing', 'Sitting', 'Sitting_left',
    'Standing', 'Bending', 'Sitting_left', 'Sitting_right', 'Bending',
In [182]: Standing_down', 'Bending', 'Bending', 'Sitting_left',
    'Standing_up', 'Sitting_right', 'Sitting_left', 'Standing_down',
with open('body_language.pkl', 'wb') as f:
    pickle.dump(fitting, f)
    model = pickle.dump(fitting, f)
    'Standing_down', 'Sitting_right', 'Sitting', 'Sitting', 'Sitting',
    'Sitting', 'Sitting_left', 'Standing_up', 'Standing_up',
In [183]: Sitting_right', 'Sitting_right', 'Bending', 'Sitting_right',
model 'Sitting', 'Standing_down', 'Sitting_right', 'Standing_down',
    'Sitting', 'Sitting_left', 'Standing_down', 'Sitting_left',
    'Sitting_left', 'Sitting_right', 'Standing_down', 'Sitting_right',
Out[183]: Sitting', 'Sitting_left', 'Sitting_left', 'Standing_up',
    'Bending', 'Bending', 'Standing_down', 'Standing', 'Sitting',
Pipeline(steps=[('standardscaler', StandardScaler()),
    ('randomforestclassifier', RandomForestClassifier())])
    'Sitting_left', 'Standing', 'Standing_down', 'Sitting_left',
    'Sitting_left', 'Standing_down', 'Sitting_right', 'Sitting_left',
In [ ]: Bending', 'Sitting', 'Standing_down', 'Sitting', 'Sitting',
    'Standing_down', 'Standing_down', 'Sitting', 'Bending',
    'Sitting_right', 'Sitting_right', 'Sitting', 'Sitting', 'Sitting',
    'Sitting', 'Bending', 'Standing_down', 'Sitting_left',
    'Sitting_left', 'Standing_up', 'Sitting_left', 'Standing',
    'Sitting', 'Sitting_left', 'Sitting_right', 'Sitting',
    'Standing_down', 'Bending', 'Sitting', 'Bending', 'Bending',
    'Sitting_right', 'Sitting', 'Sitting_right', 'Standing_down',
    'Bending', 'Sitting_right', 'Sitting', 'Standing_up',
    'Sitting_right', 'Sitting_right', 'Bending', 'Sitting',
    'Sitting_left', 'Standing_up', 'Sitting_left', 'Sitting_left',

```